



Hola a todos!

Bienvenidos a la especializacion de seguridad defensiva de Hackademy de la Ekoparty.

Para mi es un placer tenerlos aca,

Es mi primera vez siendo instructora pero tenganme paciencia o piedad.. mi intencion es que ustedes se lleven información valiosa y con que aprendan aunque sea algo nuevo, voy a estar contenta

pero bueno, me presento...



Hola!

SOY FLOR

Soy ingeniera en informática y me especializo en inteligencia artificial.

Mail: flor.p.vela@gmail.com

Github: @florvela

Repositorio de la materia:

<https://github.com/florvela/IA-y-automatizacion-en-seguridad-defensiva>

soy Flor,

soy ingeniera informática y en realidad mi especialización es inteligencia artificial, estoy trabajando ahora en una empresa de seguridad... pero bueno, mi background es más automation e Inteligencia artificial

entonces les dejo mi mail por si me equivoco y me quieren mandar un mensaje enojados, o al contrario si tienen alguna duda la idea es mantener un canal de conversación abierto

y también les dejo un repositorio que arme con la teoría y donde voy a estar poniendo los scripts que vayamos viendo en las clases.

ANTES DE EMPEZAR

"No hay que aprender todo de memoria...
Hay que saber qué googlear"

antes de empezar,

hace 10 años cuando empezaba a estudiar informática, me empecé a dar cuenta qué había mucho por aprender

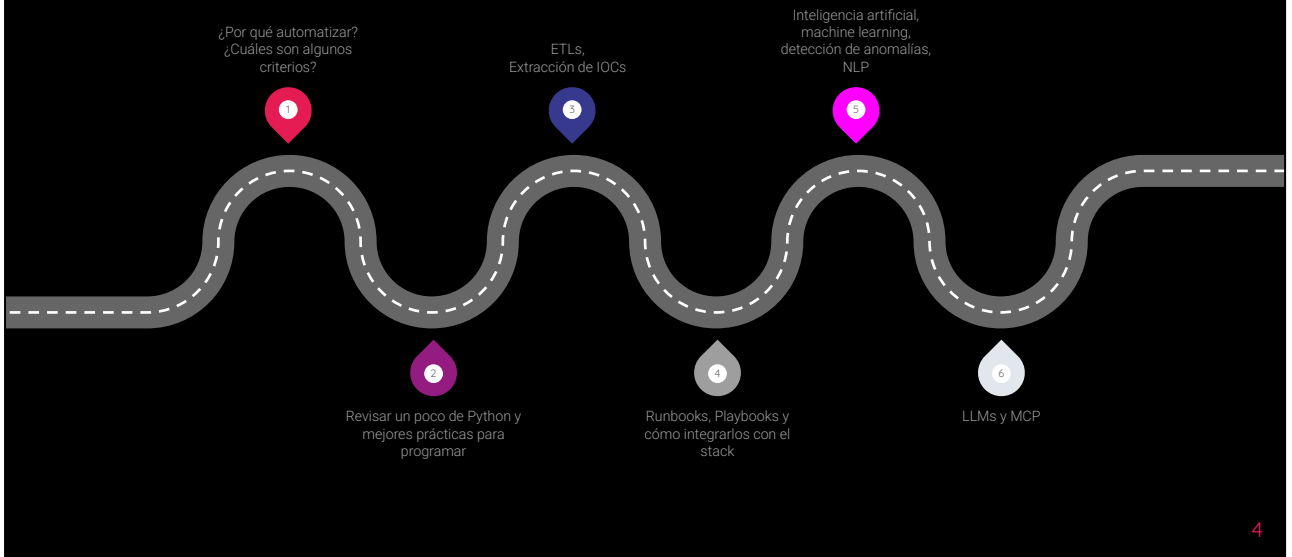
y sentía que estaba atrasada con todo... y hoy también un poco lo siento, con cómo avanza la tecnología y cada día hay un paper nuevo o nuevas arquitecturas

pero un compañero de la facultad, más grande que yo... me dijo "Flor vos no tenes qué saber todo de memoria, tenes qué saber qué cosas existen y después cuando las necesites las googleas y las lees y las entiendes"

me gustaria dejarles eso, qué quizás algunos sientan que las clases son superficiales, pero la idea es mostrar el abanico de posibilidades y que cuando tengan que implementar soluciones, sepan más o menos qué cosas existen, algunos conceptos y que sean capaces de buscar de manera individual.

por eso en estas clases quiero mantenerlo lo más general posible, y no centrarlas en herramientas.. porque por ejemplo puedo hoy sentarme a explicarte Splunk y mañana la empresa quiebra o borra todo y no te llevaste nada...

ROADMAP



En este módulo vamos a estar viendo desde lo más básico qué es entender los problemas de no automatizar, cómo resolverlos,

vamos a ver un poco de python y mejores prácticas para programar

vamos a entender ETLs y extracción de indicadores de compromiso

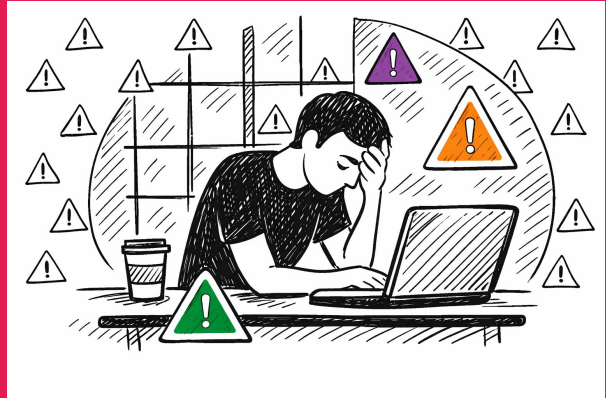
Runbooks, Playbooks,

y mi parte favorita, ML IA NLP LLMs y MCP y casos prácticos...

¿POR QUÉ AUTOMATIZAR?

Hoy vamos a ver una pequeña pero necesaria introducción... vamos a ver por qué hay que automatizar, cuando no conviene hacerlo y algunos casos de uso que se automatizan habitualmente en el SOC...

1.000 a
10.000
alertas por día



El mayor enemigo del SOC hoy en día no es si los atacantes son sofisticados.... es el volumen

volumen de información, y volumen de alertas

en los últimos años la superficie de ataque creció... Más endpoints, más usuarios remotos, más servicios cloud, más integraciones con terceros.

Cada fuente diferente genera logs.

Cada log puede generar una alerta.

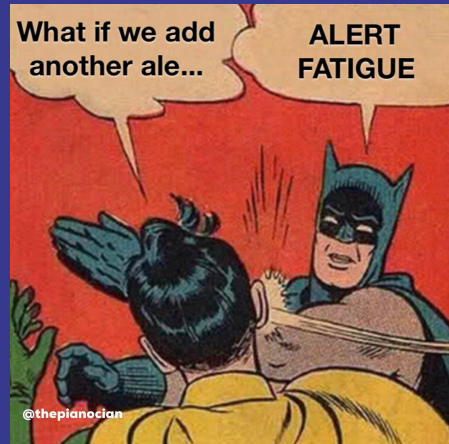
pero imagínate recibir entre **1.000 y 10.000 alertas por día**.

Encima algunos reportes de organizaciones más grandes hablan de hasta 100.000 alertas por día.

Con estas cifras revisar manualmente es imposible

¿Por qué tantas alertas?

- Reglas amplias que capturan ruido
- Falta de contextualización
- Múltiples fuentes repitiendo eventos
- Reglas obsoletas de un entorno antiguo



¿Pero por qué hay tantas alertas?

por un lado tenemos

- **Reglas de detección muy amplias:** escritas para no perderse nada, pero que capturan mucho ruido.
- **Falta de contextualización:** el SIEM no sabe si un comportamiento es normal para ese usuario o no.
- **Múltiples fuentes repitiendo eventos:** esto es qué un mismo incidente puede generar alertas del firewall, el EDR, el proxy y el SIEM al mismo tiempo. entonces vamos a tener alertas repetidas
- **Reglas obsoletas:** que están escritas para entornos que ya cambiaron y que hoy generan falsos positivos ... pero veamos qué es un falso positivo?

Falsos positivos

(falso positivo)



(falso negativo)



Un **falso positivo** es una alerta que el sistema genera pensando que hay una amenaza, pero que en realidad es actividad legítima.

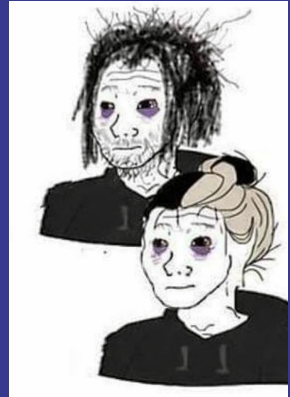
Cada falso positivo consume tiempo de análisis. Si el 90% de las alertas son falsas el analista solo dedica el 10% de su tiempo a amenazas reales.

y no solo eso... cuando todo es una alerta.. el analista ya no distingue lo urgente y surge la fatiga del analista

Fatiga del analista

Agotamiento cognitivo por
revisar miles de alertas
repetitivas sin impacto real.

- Incidentes reales pasan desapercibidos
- Se extiende el dwell time
- Baja calidad de investigaciones



La **fatiga del analista** es el agotamiento mental o cognitivo que resulta de revisar miles de alertas repetitivas con poco impacto real.

Es uno de los factores más subestimados en la seguridad operacional.

podríamos decir que algunos síntomas son:

- Cierre de alertas sin revisar correctamente ("alert dismissal").
- Disminución de la atención en horas críticas (madrugada, fin de semana).
- mucha rotación del personal en el SOC (porque el burnout es una causa frecuente de renuncia).
- Desconfianza en las herramientas: si "todo es falso positivo", se ignoran alertas reales.

y entonces las consecuencias son:

- Incidentes reales pasan desapercibidos durante horas o días.
- El dwell time (que es el tiempo que el atacante permanece en la red sin ser detectado) se extiende significativamente.
- Se afecta la calidad de las investigaciones porque el analista está sobrecargado.

NO AUTOMATIZAR ES COSTOSO

Costo humano

Reemplazar a un analista
cuesta entre 6 y 12
meses de productividad

Costo reputacional

Pérdida de confianza de
los clientes

Costo de incidentes

Daño causado

Costo regulatorio

Multas a pagar bajo
GDPR, HIPAA u otros

no automatizar es costoso

por un lado, tenemos el **Costo humano: porque si** Los analistas de SOC tienen una de las tasas de burnout más altas, y a raíz del burnout renuncian. Reemplazar a un analista de cuesta entre 6 y 12 meses de productividad perdida entre búsqueda, contratación y curva de aprendizaje.

Costo reputacional: que es difícil de cuantificar pero pasa: hablamos de la pérdida de confianza de clientes y socios después de un incidente mal gestionado y esto puede tener efectos duraderos en el negocio.

Costo de un incidente no detectado: lo que nos saldria... y el

Costo regulatorio: En sectores como finanzas o salud, una brecha no detectada a tiempo puede derivar en multas millonarias de normativas sectoriales como GDPR

Por qué automatizar es **OBLIGATORIO**

11

La automatización no es una opción de lujo en la seguridad, sino una necesidad.

Los equipos de ciberseguridad enfrentan un volumen sin precedentes de alertas, amenazas y eventos de seguridad.

y sin automatización, las organizaciones se ven atrapadas en un ciclo de respuesta manual lento, ineficiente y propenso a errores humanos.

Limitaciones del modelo manual

Velocidad de respuesta



Costo operacional



El enfoque manual (al menos en en seguridad) no escala.

por un lado....

La **velocidad manual** no alcanza frente a los ataques actuales. Un atacante puede comprometer sistemas en minutos, mientras que un analista puede tardar horas o días en detectarlo. Ese tiempo extra le da ventaja al atacante para moverse dentro de la red, robar datos y consolidar el ataque.

El **costo operativo** tampoco es sostenible. Para manejar más alertas manualmente, se necesitan más personas. Eso implica más costos de contratación, formación y retención, algo que muchas organizaciones no pueden sostener, especialmente en un contexto donde falta talento en ciberseguridad.

Tasa de error humano



La **tasa de error humano** también es un factor que afecta.

Los analistas pueden equivocarse, saltar pasos o pasar por alto alertas cuando están sobrecargados. Esto puede llevar a no detectar incidentes, tomar malas decisiones o empeorar la situación. Y cuanto más avanza el turno, digamos más tarde es, más aumenta la fatiga y los errores.

Beneficios de la automatización en el SOC

Velocidad: Respuesta instantánea

Escalabilidad: Menos costo
por más eventos

Consistencia: Procesos sin errores

Alivio para los analistas:

Menos tareas aburridas



pero cuáles son los **Beneficios de la automatización en el SOC**

El primer beneficio es la **velocidad de respuesta**. Tareas que antes tomaban minutos u horas ahora se hacen en segundos. Por ejemplo, un playbook puede aislar un equipo comprometido, recolectar evidencia, notificar y crear un ticket automáticamente en menos de un minuto...

Otro punto clave es que permite **escalar sin aumentar costos al mismo ritmo**. Un playbook puede ejecutarse miles de veces sin necesidad de más personas. Esto libera a los analistas para enfocarse en tareas más importantes, como investigar ataques reales.

También la **consistencia**. Las máquinas siempre siguen los mismos pasos... no se olvidan. no tienen variaciones. Esto evita errores humanos y nos asegura que los procesos se ejecuten correctamente cada vez.

Por último, reduce la **carga mental** del equipo. si sacamos las tareas repetitivas, los analistas pueden concentrarse en problemas más interesantes, lo que mejora la motivación y ayuda a retener talento.

Caso de uso: phishing



15

Algunos casos de automatización tienen impacto inmediato. Uno claro es la **respuesta a phishing**.

Cuando llega un email sospechoso, podemos tener un playbook que automáticamente puede: extraer direcciones, dominios y links, verificarlos contra fuentes de amenazas, bloquear o poner en cuarentena el mensaje si es malicioso, avisar al usuario y generar un caso.

Todo esto va a pasar en segundos, evitando que otros usuarios caigan en el ataque.

Caso de uso: detección y contención de malware



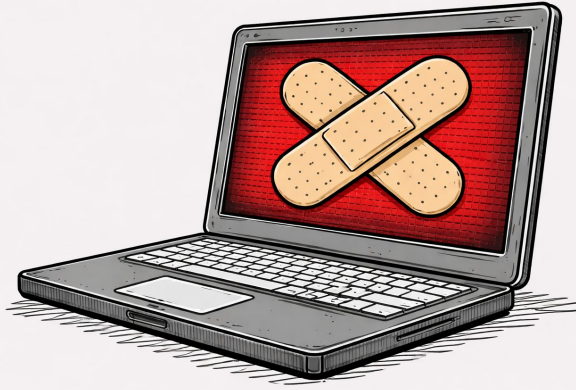
16

otro caso de uso para automatizar sería la **detección y contención de malware**

Si un equipo muestra actividad sospechosa (por ejemplo, un archivo raro ejecutándose), un playbook puede automáticamente: verificar si el archivo es malicioso, aislar el equipo de la red, recolectar evidencia, hacer un escaneo y avisar al equipo de seguridad.

y de nuevo, todo esto en segundos.

Caso de uso: Remediación de vulnerabilidades



17

otro caso de uso para automatizar sería parte de **remediación de vulnerabilidades**

Cuando aparece una vulnerabilidad, un playbook puede identificar los sistemas afectados, asignar tareas de corrección, notificar a los responsables y hacer seguimiento hasta que se apliquen los parches.

Así, un proceso que normalmente lleva semanas se puede reducir a horas.

Caso de uso: Enriquecimiento y triage de alertas



también podemos automatizar El **enriquecimiento y triage de alertas**

Cada alerta tiene que revisarse para entender si es una amenaza real o una falsa alarma. Si esa primera revisión se automatiza con reglas y consultas a otras fuentes, llega menos ruido al analista y el equipo puede enfocarse mejor en los casos realmente importantes.

Cuando **NO** automatizar

La automatización no siempre es la respuesta



La automatización no es siempre la respuesta.



En algunos casos, automatizar es peligroso.

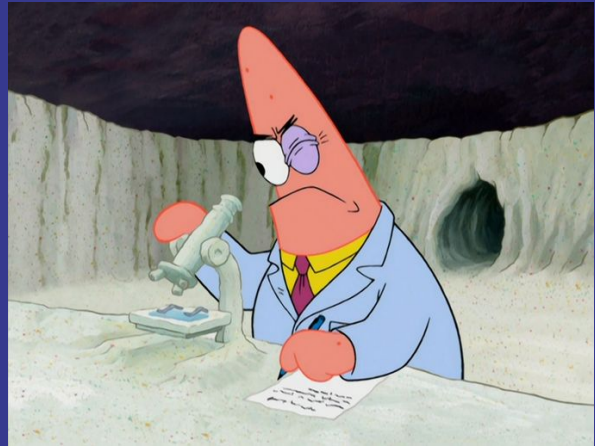
aislar incorrectamente un servidor de producción puede costar miles de dólares en downtime.

O BLOQUEAR automáticamente el acceso de un ejecutivo puede afectar el negocio.

hay que tener discernimiento sobre cuándo la automatización es contraproducente y dónde el "human in the loop" es necesario.

Decisiones que requieren juicio humano

- si tienen muchas variables subjetivas
- si afectan empleados
- si no ocurren frecuentemente
- si no hay precedentes



21

Algunas decisiones son demasiado complejas o ambiguas para automatización: por ejemplo

- **Decisiones con qué tienen muchas variables subjetivas:** "¿Esto es un comportamiento sospechoso o normal?" y va a depender del contexto, rol, departamento, hora, del año.
- **Decisiones que afectan empleados:** como "deshabilitar cuenta de usuario" afectan el trabajo de una persona real. Requiere verificación humana.
- **o decisiones de poca frecuencia...** Si esto pasa una vez por año, no vale la pena automatizar. Si ocurre 100 veces por día, sí.
- **o Incidentes sin precedentes:** Si nunca viste este patrón antes, ¿cómo vas a hacer un playbook? cómo puedes definir las reglas para automatizarlo... La máquina no puede manejar lo completamente nuevo.

Riesgo de falsos positivos en acciones bloqueantes

Supongamos que tenemos un pico de tráfico y lo confundimos con un DDoS y bajamos el servidor de nuestro sitio web de ventas... pero es "Black Friday"



22

Hay que tener precaución cuando queremos automatizar acciones bloqueantes... Porque siempre podemos tener falsos positivos.

Pero analicemos qué implica eso:

no es lo mismo tener un falso positivo y notificarlo a tener un falso positivo y generar un bloqueo... en "notificación", mandar un email diciendo "che esta alerta es sospechosa" es de bajo riesgo... pero en "bloqueo" es decir por ejemplo aislar un servidor, deshabilitar cuentas, borrar archivos... es ALTO riesgo.

Por ejemplo, podríamos generar un impacto en la disponibilidad

supongamos que tenemos un pico de tráfico y lo confundimos con un DDoS y bajamos el servidor de nuestro sitio web de ventas... pero es "Black Friday"

entonces tenemos una denegación de servicio auto infligida.

Escenarios de alto impacto y baja tolerancia al error

Servidores importantes

Si los aislás por error, podés frenar sistemas clave del negocio.

Red y sistemas centrales

Si bloqueás algo importante, puede caerse toda la operación.

Backups y acceso físico

Un error puede hacer perder datos o dejar gente sin acceso a la oficina.

veamos entonces escenarios de alto impacto y baja tolerancia al error... sistemas donde automatizar acciones bloqueantes sin revisión humana puede ser... riesgoso.

- Si se aísla por error un servidor importante, se pueden frenar procesos clave del negocio.
- Si se bloquea un componente central de la red, puede verse afectada toda la operación.
- Y si el error sucede en backups o en sistemas de acceso físico, las consecuencias pueden ser pérdida de datos o que literalmente personas no puedan entrar a la oficina.

Por eso, en estos casos, la automatización debe usarse con mucho cuidado y normalmente con aprobación humana previa o con roles bien definidos

Human-in-the-loop

Automatización completa:



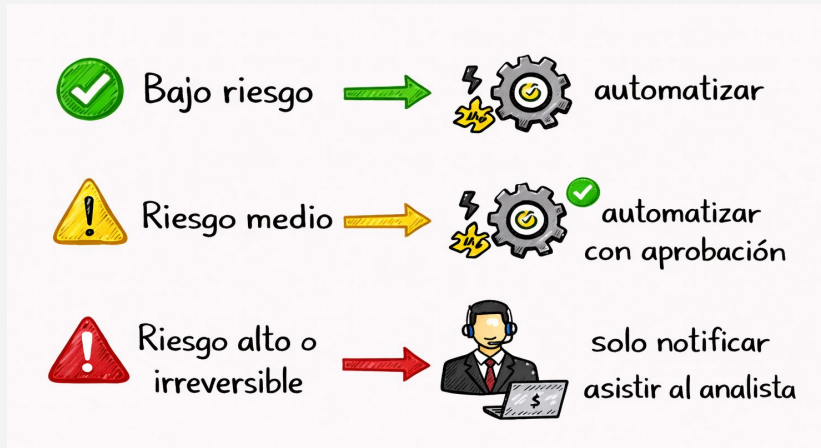
Human-in-the-loop:



Para situaciones riesgosas, siempre tenemos la opción de automatizar con “human in the loop”,

esto significa que el sistema automatiza la investigación pero requiere decisión humana antes de realizar una acción irreversible

Automatizar vs. asistir vs. notificar



Algunas acciones son seguras y repetitivas, así que conviene hacerlas automáticamente.

Otras pueden automatizarse solo con aprobación humana.

Y las más riesgosas o irreversibles no deberían automatizarse, sino quedar en manos de un analista.

La regla general es simple: **si el riesgo es bajo, se puede automatizar; si el impacto de un error es alto, tiene que intervenir una persona.**

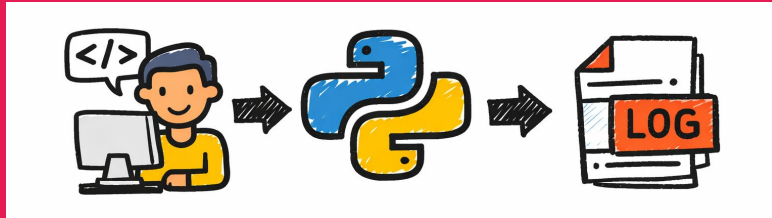


*No se trata de automatizar todo, sino
de automatizar lo correcto.*

no se trata de automatizar todo, sino de automatizar lo correcto.

Espero que este segmento haya quedado claro y que sigan disfrutando de la automatización

[FIN SEGMENTO - 16 mins]



FUNDAMENTOS DE PYTHON

27

Hola a todos! Soy Flor, bienvenidos de vuelta a la Especialización en seguridad defensiva

En esta sección vamos a ver un repaso de python y también vamos a ver buenas prácticas a la hora de programar

la idea es poner un poco las bases para poder entender código que vamos a ver más adelante...

Lo que es python no va a ser muy profundo, porque el tiempo es limitado

si les interesa aprender más del lenguaje, gracias a la tecnología... hoy aprender es más fácil que nunca y hay muchísimos recursos en internet... pero si ya saben de python, feel free para ir a la siguiente parte del módulo.

bien, si seguís acá, te comento...

¿Por qué Python?



Simplicidad y legibilidad

En comparación con otros lenguajes, necesita menos código para hacer lo mismo.



Velocidad de desarrollo

Permite trabajar con rapidez: crear scripts, probar ideas y hacer ajustes sin tanta complejidad.



Ecosistema de librerías

Ya existen herramientas listas para usar para conectarse a APIs, procesar datos, trabajar con servidores, cifrado, etc.



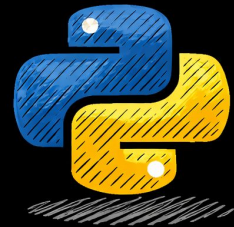
Integración seamless

Se integra bien con muchas plataformas y herramientas que ya se usan en ciberseguridad.



Talento disponible

Muchos analistas ya tienen experiencia con Python. Scripts más fáciles de mantener y entender por otros en el equipo.



28

Por qué Python

Python se usa mucho porque es **fácil de leer y escribir**. En comparación con otros lenguajes, necesita menos código para hacer lo mismo, y eso lo vuelve más práctico para analistas que no son desarrolladores de tiempo completo, que no se dedican a programar.

También permite trabajar con mucha **rapidez**. muchas veces hace falta resolver algo ya, y Python ayuda a crear scripts, probar ideas y hacer ajustes sin tanta complejidad.

Otro punto fuerte es que tiene muchas **librerías útiles**. Ya existen herramientas listas para usar para conectarse a APIs, procesar datos, trabajar con servidores, cifrado, detección de malware y otras tareas comunes.

Además, se integra bien con muchas **plataformas y herramientas** que ya se usan en ciberseguridad, por lo que sirve muy bien para conectar sistemas entre sí.

Por último, mucha gente ya conoce Python. Eso hace que los scripts sean más fáciles de mantener y entender por otras personas del equipo.

Librerías esenciales para seguridad

requests

Consumir cualquier API. VirusTotal, Crowdstrike, Splunk... todo con 2 líneas.

```
import requests
resp = requests.get(url, headers={"x-apikey": KEY})
data = resp.json()
```

hashlib

Calcular hashes criptográficos (MD5, SHA256). Huellas digitales de archivos.

```
import hashlib
sha256 = hashlib.sha256(bytes).hexdigest()
```

paramiko

SSH automático. Ejecutá comandos en servidores remotos para contención

```
ssh.exec_command("ps aux | grep malware")
```

datetime

Manipular timestamps. Parsear formatos distintos. calcular duraciones.

```
from datetime import datetime, timedelta
hace_24h = datetime.now() - timedelta(days=1)
```

Una de las grandes ventajas de Python en seguridad es que ya existen **muchas librerías listas para usar** para casi cualquier tarea. Eso evita tener que construir todo desde cero.

Por ejemplo:

- **requests** sirve para conectarse a APIs. y es útil porque muchas herramientas de seguridad, como VirusTotal, Splunk o servicios cloud, se consultan por API.
- **hashlib** sirve para calcular hashes de archivos, por ejemplo SHA256. Esto permite identificar archivos y compararlos con bases de datos de amenazas.
- **paramiko** sirve para conectarse por SSH a máquinas remotas y ejecutar comandos desde un script.
- **datetime** sirve para trabajar con fechas y horarios, algo clave para entender los logs

REGEX

Expresiones regulares.

```
import re

log_line = "2024-03-18 14:32:15 User admin logged in from 192.168.1.50"

# Extraer IP
match = re.search(r'\d+\.\d+\.\d+\.\d+', log_line)
if match:
    ip = match.group()
    print(f"IP encontrada: {ip}")
```

30

Las **expresiones regulares** sirven para buscar y extraer patrones dentro de un texto. Esto es muy útil en seguridad porque muchos logs no vienen perfectamente ordenados, sino como grandes bloques de texto donde la información importante está mezclada.

Con regex, en vez de buscar una palabra exacta, podés buscar un **patrón**. Por ejemplo, una dirección IP, un hash, un email o un comando.

Son especialmente útiles para sacar datos concretos de logs que no están bien estructurados.

BUENAS PRÁCTICAS para scripts de automatización

Credenciales - modularidad - logging - testing - resiliencia



31

Un script que funciona una vez no es suficiente. Lo importante es que funcione de manera confiable, que no filtra credenciales, que avisa cuando falla, que puede ser mantenido por otro analista y testeado antes de ir a producción. Vamos a ver las prácticas que separan el scripting improvisado de una solución profesional

GESTIÓN SEGURA DE CREDENCIALES

- | | |
|---------------------------------|---------------------------------------|
| ✓ Usa variables de entorno | ✗ Codificar credenciales en el código |
| ✓ Utiliza un gestor de secretos | ✗ Enviar archivos .env |
| ✓ Escanea antes de hacer commit | ✗ Guardar secretos en el código |

✗ NUNCA hagas esto

```
api_key = "sk_live_1234567890abcdefghijklmnopqrstuvwxyz"
password = "SuperSecure123!@#"
# Cualquier acceso al repo = credenciales expuestas
```

✓ Variables de entorno

```
import os
api_key = os.getenv("VT_API_KEY")
if not api_key:
    raise ValueError("No configurada")
```

✓ Archivos .env

```
# .env (NUNCA en git)
VIRUSTOTAL_API_KEY=sk_live_...
SPLUNK_HOST=splunk.company.com

from dotenv import load_dotenv
load_dotenv()
```

Gestión segura de credenciales

En automatización, las credenciales nunca tienen que estar escritas directamente en el código. Si una API key, usuario o contraseña queda en un script, cualquier persona que acceda al archivo o al repositorio puede verla. Además, si se sube por error a git, puede quedar expuesta aunque después se borre. Por eso, las credenciales no tienen que estar en el código, en comentarios, en logs ni en archivos versionados.

La forma más común de manejar esto es usar **variables de entorno** o un archivo **.env** que no se sube al repositorio. En entornos más grandes, se usan **vaults de secretos**, que guardan las credenciales de forma centralizada y segura. La idea es siempre la misma: que el script pueda usar la credencial, pero que no esté visible en el código.

MODULARIDAD

Scripts monolíticos

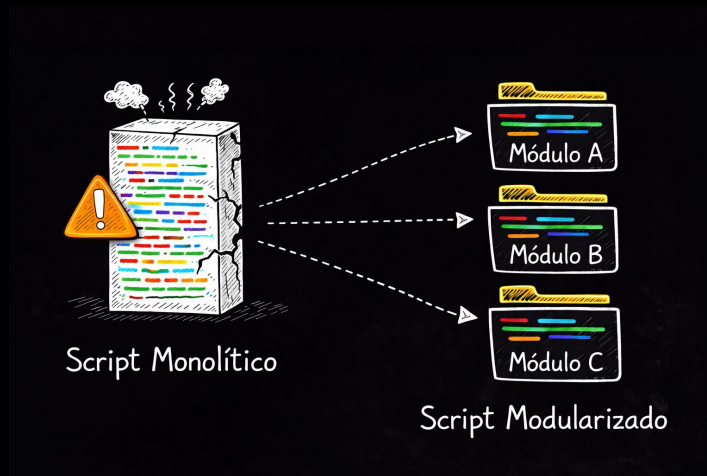
- Todo en un solo archivo
- Difícil de testear, reutilizar y debuggear.



Script Monolítico

El punto de partida de la mayoría es escribir todo en un solo archivo, de arriba hacia abajo. Funciona al principio. Después de 200 líneas, se convierte en un caos para debuggear, imposible de testear, difícil de reutilizar, una sola falla rompe todo.

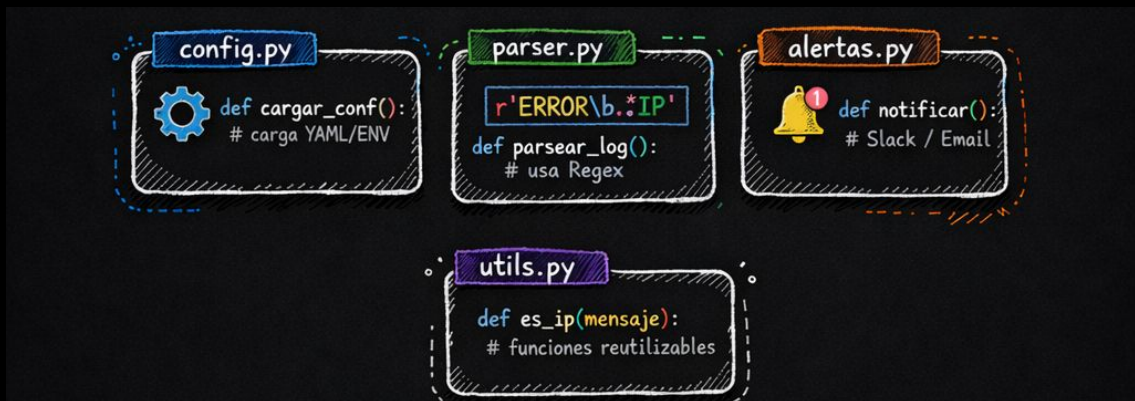
MODULARIDAD



Para proyectos más grandes (por ejemplo un conjunto de playbooks, un pipeline de enriquecimiento), queremos modularidad que

la modularidad implica separar en archivos y en funciones o clases

MODULARIDAD



35

Cada concepto vive en su propio módulo: todas las integraciones con VirusTotal en un archivo, todos los parsers de logs en otro, todas las acciones de respuesta en otro.

Cada archivo importa lo que necesita de los demás.

la idea es después consumir esos módulos desde el código... por ejemplo el playbook de phishing, puede llamar a una función `enriquecer_ip` con `virustotal` y recibir el resultado, sin saber cómo funciona el módulo de virus total internamente

LOGGING



```
logger.debug("Detalle interno - solo para desarrollo")
logger.info("Evento normal del flujo: 'inicio de análisis', 'alerta procesada'")
logger.warning("Situación inesperada pero no crítica: 'hash no encontrado'")
logger.error("Error que impidió completar una operación: 'fallo API'")
logger.critical("Error que rompe el sistema completo: 'base de datos no disponible'")
```

Python

36

otro tema importante es el logging...

La función `print()` es para desarrollo local donde vos estás mirando la pantalla. En producción, tu script se ejecuta desde cron, de madrugada, sin nadie mirando. Si algo sale mal, `print()` desaparece en la nada.

En producción necesitás: saber **cuándo** ocurrió cada evento, poder **filtrar** por nivel de severidad, guardar logs en **archivos**

Python tiene una librería estándar que se llama **logging** que cubre todo esto. Lo configurarás una sola vez, y después usás `logger.info()`, `logger.error()`, etc. Python se encarga automáticamente de timestamps y niveles.

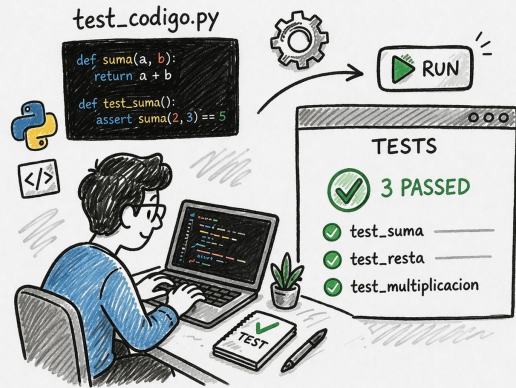
TESTING

- Tests unitarios con pytest
- Mocking

```
def test_parsear_resultado_malicioso():
    resultado = parsear_resultado_vt(RESP_VT_MALO)
    assert resultado["malicious"] == 47
    assert resultado["es_malicioso"] == True

def test_decidir_accion_aislar():
    r = {"malicious": 10}
    accion = decidir_accion(r, umbral=3)
    assert accion == "AISLAR_INMEDIATAMENTE"

# Correr tests:
# pytest tests/ -v
```



37

testing...

Por qué los scripts necesitan tests

Los scripts del SOC pueden tomar acciones importantes, como aislar equipos, bloquear IPs o crear casos. Si tienen un error, pueden afectar al sistema equivocado o dejar pasar una amenaza real. Por eso, los tests son importantes: ayudan a comprobar que el script funciona bien antes de usarlo en producción.

tenemos diferentes tipos de tests ,por ejemplo

Tests unitarios con **pytest**

que **pytest** es una herramienta de Python para hacer tests de forma simple. Un test verifica que una parte del código haga lo que debería hacer. La idea es correr esos tests cada vez que cambiás algo, para asegurarte de no romper el script.

y después tenemos el concepto de **Mocking: que es testear sin usar algunas funciones reales**

En los tests no conviene llamar a APIs reales, porque eso puede ser lento, gastar cuota o fallar si el servicio no responde. Para eso se usa **mocking**, que simula la respuesta de la API. Así podés probar distintos casos simulados, como una respuesta maliciosa, vacía o con error, sin depender de servicios externos.

RESILIENCIA

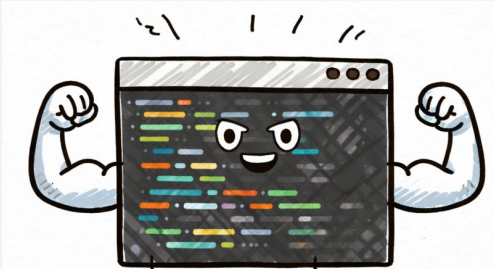
≡ Resiliencia de Código ≡



Las APIs fallan, las redes se caen temporalmente, los límites de rate se alcanzan. Un script no resiliente falla una sola vez y se queda parado para siempre — eso se convierte en un incidente de disponibilidad.

La solución es agregar **resiliencia**: lógica que reintenta automáticamente, usa fuentes alternativas, y solo escala a un humano cuando realmente todo falló.

RESILIENCIA



```
def reintentar(max=3, espera=5):
    def decorador(func):
        def wrapper(*args, **kwargs):
            for i in range(1, max + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if i == max: raise
                    time.sleep(espera)
            return wrapper
        return decorador

@reintentar(max=3, espera=10)
def consultar_virustotal(hash):
    ... # reintentar automáticamente
```

Una de las formas más elegantes es usar un **decorador** que envuelve una función y la reintenta si falla

Un **decorador en Python** es una función que recibe otra función y devuelve una nueva función con comportamiento modificado.

La idea clave es: **envolver una función para agregar lógica sin cambiar su código original.**

Veamos un ejemplo

Logs → Datos estructurados → Detecciones → Decisiones

Código en [github](#)

Código en [Google Colab](#)

Veamos un ejemplo de estas buenas prácticas y cómo podemos usar python para ir facilitando el parseo de logs

Aca tenemos dos links, uno a Google Colab, qué es un entorno de Google para poder ejecutar jupyter notebooks en la nube, está bueno porque nos olvidamos de instalar dependencias y librerías y podemos ir jugando

y el otro link es a Github, qué es el mismo notebook, pero por si quieren descargarse el código de manera local..

1. primero vamos a tener los imports, donde importamos las librerías qué vamos a estar usando, comenzando por RE = REGEX



*Un script útil resuelve un problema.
Un buen código evita crear uno nuevo.*

41

*Un script útil resuelve un problema.
Un buen código evita crear uno
nuevo.*

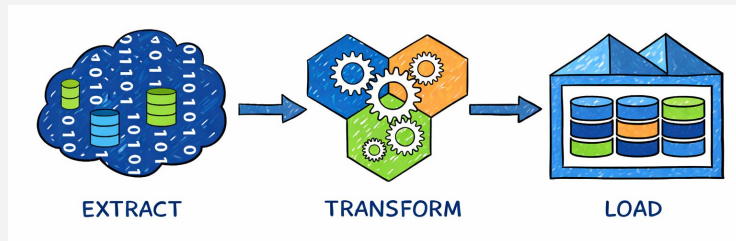
Vimos por qué Python es una herramienta clave, repasamos sus bases y entendimos que escribir código no alcanza: tiene que ser claro, seguro y confiable. Estas prácticas son las que transforman un script en una solución lista para producción

Espero que este segmento haya quedado claro y que sigan disfrutando de la automatización

PIPELINES ETL E IOCS

Hola a todos! Soy Flor, bienvenidos de vuelta a la Especialización en seguridad defensiva
y hoy vamos a ver pipelines ETL e indicadores de compromiso

¿QUÉ ES ETL?



ETL viene de las siglas de Extract Transform y Load

que es el proceso de: extraer datos de fuentes heterogéneas (es decir diferentes), transformarlos y cargarlos en diferentes destinos.

¿QUÉ ES ETL?



44

Sin ETL, cada herramienta del SOC entrega la información de una forma distinta. El SIEM, el EDR y el firewall pueden registrar el mismo incidente, pero con formatos diferentes. Si esos datos no se transforman a un esquema común, compararlos y correlacionarlos va a ser muy difícil.

Es como comparar precios en tres supermercados donde uno usa pesos, otro dólares y otro euros. Hasta que no convertís todo a la misma moneda, no sabés realmente cuál es más caro o más barato.

En seguridad, transformar a un esquema común es todavía más importante porque se manejan **muchísimos eventos**,

hay que procesarlos **rápido**,

los formatos son **muy distintos entre herramientas**, y además no se puede perder información.

Un error en ese proceso puede hacer que se pase por alto una amenaza o que perdamos evidencia útil para investigar un incidente.

EXTRACT



Fuentes de datos del SOC

- archivos de logs locales
- APIs de SIEM/EDR
- Colas de mensajes (SQS / Kafka)
- Data Lakes (s3 / parquet)

La fase de **Extract** es la parte donde el pipeline obtiene los datos desde las distintas fuentes del SOC. Esos datos pueden venir de:

- archivos locales,
- APIs por ejemplo de un SIEM o EDR,
- de colas de mensajes SQS o
- almacenamiento en la nube cómo un S3 de AWS.

El problema es que cada fuente se accede de una manera distinta y entrega la información en formatos diferentes. Por eso, esta etapa tiene que poder traer datos de forma confiable, sin importar de dónde vengan.

TRANSFORM



Convertir ruido en datos utilizables

- parseo
- normalización
- enriquecimiento
- deduplicación

La etapa de **Transform** es donde los datos se ordenan y se vuelven realmente útiles. Acá se toma la información que llega en distintos formatos y se convierte en el esquema común, para que sea más claro y fácil de analizar.

tenemos diferentes maneras de transformar estos datos, por lo general lo hacemos a través del

- parseo
- normalización
- enriquecimiento
- deduplicación

TRANSFORM: PARSEO

Objetivo: Ordenar datos



47

El parseo es el primer paso del Transform.

el objetivo es agarrar logs que vienen como texto y separar sus partes para convertirlos en datos más ordenados.

Parsear es **leer un formato crudo y convertirlo en campos claros**. Esto es importante porque, mientras el log siga siendo solo texto, es mucho más difícil buscar, filtrar o correlacionar eventos después.

para esto podemos usar regex cómo vimos antes, pero también puede ser que ya vengan en un formato cómo un diccionario o un JSON y la idea es conservar esas variables.. veamos un poco un par de ejemplos más claros:

TRANSFORM: PARSEO

Ejemplo: parsear evento windows

```
{
  "@timestamp": "2024-03-15T08:42:17.503Z",
  "winlog": {
    "event_id": 4625,
    "channel": "Security",
    "computer_name": "DESKTOP-ABC123",
    "provider_name": "Microsoft-Windows-Security-Auditing",
    "record_id": 8821043,
    "task": "Logon",
    "keywords": ["Audit Failure"],
    "user": {
      "name": "jsmith",
      "domain": "CORP",
      "type": "User"
    },
    "event_data": {
      "LogonType": "3",
      "FailureReason": "Unknown user name or bad password",
      "IpAddress": "192.168.10.45",
      "IpPort": "51322"
    }
  },
  "process": { "name": "lsass.exe", "pid": 748 },
  "host": {
    "os": { "name": "Windows Server 2019", },
    "ip": ["10.0.1.15"]
  },
  "message": "An account failed to log on. Subject: SYSTEM. Logon Type: 3. Account: jsmith.",
  "agent": {
    "version": "8.12.0",
    "type": "winlogbeat"
  }
}
```

↓ parsear_windows_event(evento_json) ↓

SALIDA — DICCIONARIO NORMALIZADO **PARSEADO**

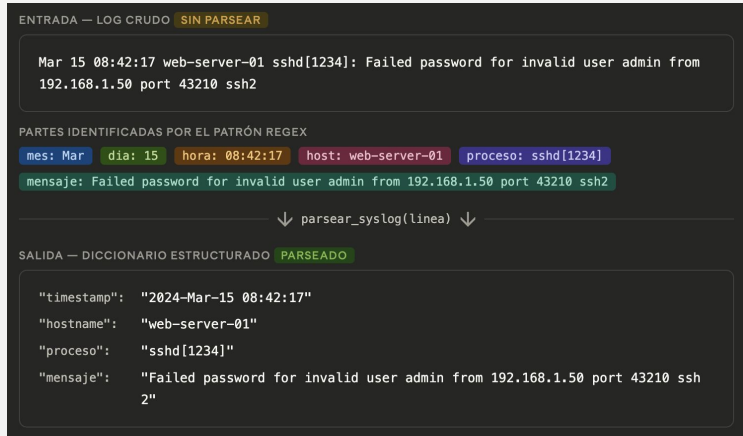
```
{
  "timestamp": "2024-03-15T08:42:17.503Z",
  "event_id": 4625,
  "channel": "Security",
  "computer": "DESKTOP-ABC123",
  "usuario": "jsmith",
  "proceso": "lsass.exe",
  "mensaje": "An account failed to log on. Subject: SYSTEM. Logon Type: 3. Account: jsmith.",
  "raw": { ... objeto JSON completo original ... }
}
```

El campo "raw" conserva el JSON original completo — útil para enrichment posterior sin reparsear.

- En un evento de Windows se pueden extraer campos como el usuario, el equipo o el ID del evento.

TRANSFORM: PARSEO

Ejemplo: de una línea de syslog sacar la fecha, el host, el proceso y el mensaje.



ENTRADA — LOG CRUDO SIN PARSEAR

```
Mar 15 08:42:17 web-server-01 sshd[1234]: Failed password for invalid user admin from 192.168.1.50 port 43210 ssh2
```

PARTES IDENTIFICADAS POR EL PATRÓN REGEX

mes: Mar dia: 15 hora: 08:42:17 host: web-server-01 proceso: sshd[1234]
mensaje: Failed password for invalid user admin from 192.168.1.50 port 43210 ssh2

↓ parsear_syslog(linea) ↓

SALIDA — DICCIONARIO ESTRUCTURADO PARSEADO

```
{
  "timestamp": "2024-Mar-15 08:42:17",
  "hostname": "web-server-01",
  "proceso": "sshd[1234]",
  "mensaje": "Failed password for invalid user admin from 192.168.1.50 port 43210 ssh2"
}
```

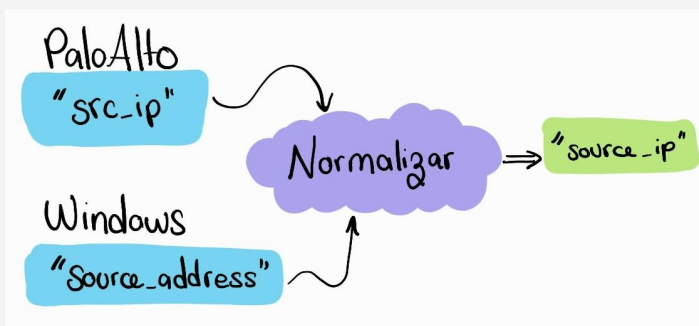
Por ejemplo, de una línea de syslog se puede sacar la fecha, el host, el proceso y el mensaje.

[y fíjense que en este ejemplo el timestamp no solamente se extrajo si no que se cambio de formato]

TRANSFORM: NORMALIZACIÓN

Objetivo: Convertir a esquema común

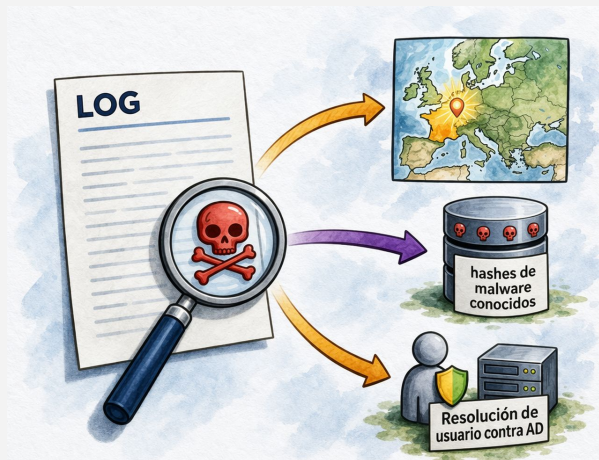
Estándares de la industria: ECS y OCSF



Una vez parseado, necesitás convertir cada evento a un esquema normalizado. Esto significa que sin importar si viene de Palo Alto, Windows o CrowdStrike, los campos clave se llaman siempre igual. Palo Alto llama "src_ip", Windows podría llamarlo "source_address": normalizás los dos a "source_ip". Esto hace que después, en la fase de Load o en tu análisis, podés escribir reglas una sola vez que funcionan para todos los vendedores.

TRANSFORM: ENRIQUECIMIENTO

Objetivo: agregar contexto



51

Enriquecimiento es agregar contexto

Cuando un evento ya está normalizado, todavía puede faltarle contexto. Por ejemplo, una IP sola dice poco. Enriquecer un evento significa agregar información útil, cómo:

- si esa IP es interna o externa,
- la geolocalización para ver de qué país viene,
- ver a qué red pertenece o si aparece en una lista de amenazas.

Esto hace que el evento sea mucho más fácil de entender para el analista y priorizar. Un detalle importante es que, si consultas una API externa para cada IP, el proceso se vuelve lento. Por eso suele usarse **cache**: si una IP ya fue consultada antes, se reutiliza ese resultado en vez de volver a pedirlo.

TRANSFORM: FILTRADO Y DEDUPLICACIÓN

Objetivo: eliminar lo que no nos interesa y los duplicados



52

En un SOC grande, millones de eventos por hora es normal. No todos son interesantes. El ruido es abrumador: tráfico de monitoreo, health checks, eventos de baja severidad de sistemas conocidos.

Queremos eliminar los que sabemos que no importan y para esto tenemos el filtrado.

otro concepto es la deduplicación qué es que si el mismo evento fue generado 1000 veces en el último minuto (por ejemplo, porque un sistema está en loop), querés guardar solo uno.

podemos hacer deduplicacion muy simple tomando en cuenta el mismo IP, mismo puerto, misma acción o según reglas específicas del vendor

LOAD

Llevar los logs a donde deberían estar



53

Una vez transformados y enriquecidos, los eventos van a múltiples destinos.

Algunos van al SIEM (Elasticsearch, Splunk) para búsqueda en tiempo real.

Otros van a un data lake en S3 o formato Parquet para análisis histórico.

Los eventos críticos van directamente al SOAR para crear casos.

El Load es simple comparado con el Transform, pero es crítico hacerlo de forma eficiente: no querés que un fallo en la carga deje eventos sin procesar.

ORQUESTACIÓN

Correr el pipeline automáticamente

- cronjob

```
crontab: */5 * * * * python /opt/soc/pipeline_etl.py >> /var/log/etl.log 2>&1
```

- loop continuo

```
import time

def correr_pipeline_continuo(intervalo_segundos=60):
    logger.info("Pipeline ETL iniciado en modo continuo")
    while True:
        try:
            n = pipeline_etl_soc(FUENTE_CONFIG, DESTINO_CONFIG)
            logger.info(f"Ciclo completado: {n} eventos procesados")
        except Exception as e:
            logger.critical(f"Pipeline falló: {e}")
            time.sleep(intervalo_segundos)
```

- Orquestador de workflows para pipelines complejos

54

Un pipeline ETL de seguridad no se ejecuta manualmente: corre de forma continua o programada. Una vez que lo escribís, querés que funcione sin intervención. Hay tres patrones principales, según la complejidad:

Opción 1: cron job simple

Opción 2: loop continuo con intervalo

Opción 3: Para organizaciones más maduras, un orquestador de workflows permite dependencias entre tareas, reintentos, alertas de fallo, etc.



Los datos no se vuelven útiles cuando los recolectás, si no cuando los entendés.

55

Los datos no se vuelven útiles cuando los recolectás, sino cuando los entendés

Un buen pipeline ETL no solo mueve datos: los ordena, les da contexto y permite detectar lo importante en medio del ruido. Es la base para detección, análisis y respuesta en cualquier entorno de seguridad.

[11 minutos]

EXTRACCIÓN DE IOCS

Regex - scripts - pipelines de enriquecimiento



56

ahora veamos cómo extraer IOCs, o indicadores de compromiso, y por qué son una pieza clave en el análisis

Muchas veces la información que recibimos no viene limpia ni estructurada: puede ser un mail , un log, o un reporte lleno de texto.

El desafío es pasar de ese contenido desordenado no solamente a estructurarlo si no también a

señales concretas que podamos analizar.

¿QUÉ SON LOS IOCS?

Indicator of Compromise (IOC)

Dato técnico que indica que un sistema fue o podría estar comprometido. Es un artefacto observable y verificable.



57

Empecemos definiendo un IOC (Indicator of Compromise) o indicador de compromiso

Un **IOC** es una señal concreta de que un sistema pudo haber sido comprometido. Es un dato técnico que se puede observar y verificar, como una IP, un dominio, una URL o el hash de un archivo malicioso.

Estos indicadores suelen aparecer en

- correos de phishing,
- reportes de threat intelligence,
- logs o
- análisis de malware.

Encontrarlos a mano puede ser lento y fácil de errar, por eso muchas veces se extraen automáticamente con Python.

ALGUNOS PATRONES REGEX PARA IOCS

IPv4

```
r"\b(?:25[0-5]|2[0-4]\d|[01]?\d\d?)\.{3}\b"
```

Hash MD5

```
r"\b[0-9a-fA-F]{32}\b"
```

Hash SHA1

```
r"\b[0-9a-fA-F]{40}\b"
```

Hash SHA256

```
r"\b[0-9a-fA-F]{64}\b"
```

URL

```
r"https?://[a-zA-Z0-9\._~:/?#\[\]@!$&'()*+,-;%]+"
```

Email

```
r"\b[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}\b"
```

CVE

```
r"\bCVE-\d{4}-\d{4,7}\b"
```

MITRE ATT&CK

```
r"\bT\d{4}(?:\.\d{3})?\b"
```

Para extraer estos indicadores, solemos usar expresiones regulares o regex:

Acá lo importante no es memorizar las regex, sino entender la idea. Cada tipo de IOC suele tener una forma o estructura bastante predecible, y eso nos permite buscarlo automáticamente dentro de un texto.

Si recibo un log, un mail de phishing o un reporte de threat intel, Python puede recorrer ese contenido y encontrar IPs, hashes, URLs, emails, CVEs o técnicas de MITRE sin que yo tenga que hacerlo a mano.

Esto nos ahorra tiempo, reduce errores manuales y nos permite pasar del texto crudo a un análisis automatizado.

CASOS DE USO PARA EXTRACCIÓN DE IOCS

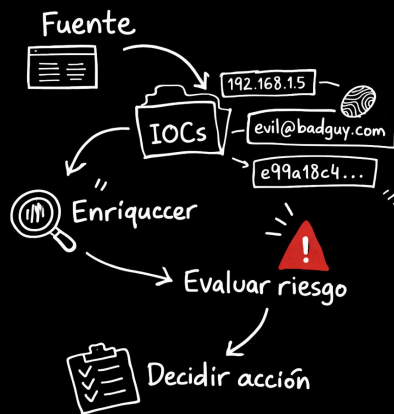
- Correos de phishing
- Alertas del SIEM/EDR
- Reportes de threat intelligence
- Análisis de malware
- Logs de red, firewall o proxy



Algunos casos de uso serian por ejemplo:

- correos de phishing para extraer links, dominios, emails, o archivos adjuntos sospechosos para poder investigarlos despues
- sacar IPs hashes o dominios de las alertas para hacer el enriquecimiento automático
- o sacar datos de reportes de threat intelligence

EXTRAER IOCS + ENRIQUECER



La extracción de IOCs no es el objetivo final, sino el primer paso del análisis.

Una vez que encontrás IPs, dominios, hashes o URLs, el siguiente paso es enriquecerlos para entender si realmente representan una amenaza.

Por ejemplo, después de extraer los IOCs de un texto o una alerta, podés consultarlos en una API o en una base de datos de threat intelligence para ver si son maliciosos.

Con esa información, podés calcular un nivel de riesgo y decidir si el caso debe escalarse, investigarse o simplemente quedar en monitoreo.

librerías como **ioc-finder**

librerías externas	regex
 Conveniente	 Control total
 Rápido de implementar	 Personalizable
 Bien testado	 Sin dependencias externas
 Menos control sobre la lógica	 Más trabajo inicial

<https://pypi.org/project/ioc-finder>

```
pip install ioc-finder
```

61

Escribir y mantener regex para cada tipo de IOC es práctico para control total, pero es tedioso y propenso a errores.

Para proyectos que necesitan extracción más robusta sin mantener los regex manualmente,

existe librerías como **ioc-finder**. Esta librería ofrece patrones ya testados y mantenidos por la comunidad,

y también tiene features extra como por ejemplo detección de direcciones Bitcoin.



La extracción de IOCs es el primer paso para transformar texto en señales accionables

La extracción de IOCs es el primer paso para transformar texto en señales accionables. Nos permite automatizar tareas repetitivas, reducir errores y preparar los datos para enriquecimiento y análisis.

Veamos un ejemplo

Construyamos un pipeline ETL

Código en [github](#)

Código en [Google Colab](#)

Bueno, ahora que vimos la teoría, vamos a construir el pipeline paso a paso en el notebook.

FUNDAMENTOS DE SOAR

Hola a todos! Soy Flor, bienvenidos de vuelta a la Especialización en seguridad defensiva
y hoy vamos a estar viendo fundamentos de SOAR

RUNBOOK VS PLAYBOOK

RUNBOOK

Texto que describe pasos — lo consulta el analista

PASO 1: Obtener información del incidente

- Registra la IP origen
- Registra el usuario atacado
- Registra los intentos fallidos

PASO 2: Verificar si la IP es conocida

- Busca en CMDB e historial

PASO 3: Tomar acciones de mitigación

- Bloquea la IP en el firewall
- Fuerza cambio de contraseña

PASO 4: Documentar y escalar

- Crea ticket y notifica equipo



PLAYBOOK

Código ejecutable — corre solo en 2-3 segundos

```
def responder_bruta_rdp(ip, usuario, intentos):
    incidente = crear_caso("RDP BruteForce",
                          ip=ip, usuario=usuario)

    # ¿IP conocida? → falso positivo probable
    if consultar_cmdb(ip):
        return "Falso positivo - IP conocida"

    # Ejecutar acciones automáticamente
    bloquear_ip_en_firewall(ip)
    forzar_cambio_password(usuario)

    # Escalar si hay acceso exitoso
    if historial_tiene_acceso(usuario):
        incidente.prioridad = "CRITICAL"

    crear_ticket_jira(incidente)
    return incidente # En 2-3 segundos ✓
```

Los mejores playbooks tienen "human-in-the-loop"

65

Cuando trabajamos en un SOC, no alcanza con saber qué hacer: necesitamos procesos claros, repetibles y, en lo posible, automatizados.

Para eso existen los runbooks y los playbooks...

un **runbook** es una guía escrita con pasos a seguir para qué el analista sepa qué hacer....

un **playbook** es ese mismo proceso qué definio el runbook, pero convertido en algo ejecutable, que podemos ejecutar dentro de un SOAR.

Es decir, el runbook dice qué hacer, y el playbook lo hace.

En código, cada paso del runbook se transforma en funciones o bloques lógicos

Aun así, no todo se automatiza por completo. cómo vimos antes, en las decisiones más delicadas, suele haber **human-in-the-loop**: que el playbook junta información, hace validaciones y después le deja al analista la decisión final

Esto es muy útil para reducir falsos positivos, porque en vez de investigar cada alerta desde cero durante horas, el playbook hace gran parte del trabajo y el analista decide con más contexto.

SOAR

Security Orchestration, Automation and Response

66

Este tipo de automatización y orquestación dentro de un SOC tiene un nombre: SOAR

SOAR significa **Security Orchestration, Automation and Response**.

Es una plataforma que ayuda a los equipos de seguridad (Blue Team / SOC) a **organizar, automatizar y ejecutar la respuesta a incidentes**.

- **Orchestration (Orquestación)** → es conectar herramientas entre sí (SIEM, EDR, firewalls, APIs, etc.)
- **Automation (Automatización)** → ejecutar tareas repetitivas automáticamente (enriquecer IPs, consultar APIs, clasificar alertas)
- **Response (Respuesta)** → permite actuar ante incidentes (bloquear una IP, aislar un endpoint, abrir un ticket)

CICLO DE VIDA DE INCIDENTE



67

Veamos la idea general del ciclo de vida de un incidente automatizado en un SOAR.

Primero llega la **detección**, es decir, la alerta entra desde herramientas como el SIEM, el EDR o el firewall. Después viene el **triaje automático**, donde el sistema junta más contexto y calcula un nivel de riesgo.

Con esa información, el SOAR toma una **decisión**: si el riesgo es bajo, puede cerrar o resolver automáticamente; si es medio, lo deja para revisión humana; y si es alto, puede disparar

una **respuesta automática**. Esa respuesta depende del caso: bloquear un phishing, aislar un endpoint que sospechamos que tiene malware o bloquear una IP en un intento de fuerza bruta.

Si el caso no se cerró solo, pasa a **revisión humana**, donde el analista valida lo que hizo el sistema y decide si aprueba, ajusta o continúa la investigación.

Y finalmente está el **cierre**, donde se documenta lo ocurrido y se aprovecha el incidente para mejorar reglas, playbooks y procesos.

Tipos de plataformas SOAR



El mercado SOAR cambia constantemente: por eso, más que memorizar herramientas, lo importante es entender las **categorías** y los **criterios** para evaluar cualquier opción que exista cuando lo necesites:

por un lado tenemos plataformas SOAR **Comerciales ... que son de nivel empresarial**

son plataformas maduras, con muchas integraciones listas para usar y soporte del proveedor. Funcionan bien, pero suelen ser caras y después cuesta salir de ese ecosistema.

Sirven para: empresas con presupuesto y muchos sistemas para integrar.

después tenemos plataformas Cloud-native

Qué son soluciones que ya vienen dentro de una plataforma más grande, como un SIEM o un XDR. Se integran muy bien con herramientas del mismo proveedor, pero suelen ser menos flexibles con otras.

Sirven para: organizaciones que ya usan casi todo del mismo vendor.

y también hay plataformas open source y autohosteadas

No tienen costo de licencia y dan más control sobre los datos y la plataforma. Pero requieren quizás un equipo más técnico que pueda instalarlas, mantenerlas y

Las plataformas open source son interesantes para aprender SOAR porque podés ver exactamente cómo funcionan por dentro.

algunos ejemplos serian Shuffle, TheHive con Cortex, Iris, OpenCTI

INTEGRACIÓN CON EL STACK

El objetivo es integrar las herramientas de seguridad y orquestar la respuesta entre ellas



La idea del SOAR no es solo correr código, si no poder integrarse con herramientas y orquestar la respuesta entre ellas... veamos algunas herramientas que podríamos integrar...

INTEGRACIÓN CON EDR

Queremos:

- Recibir alertas del endpoint.
- Consultar historial del dispositivo.
- Aislar endpoints comprometidos.
- Ejecutar acciones remotas



70

Tenemos el EDR (Endpoint Detection & Response) que es el ojos y oídos en los endpoints.

Herramientas que detectan actividad maliciosa en máquinas.

Queremos poder:

- Recibir alertas del endpoint.
- Consultar historial del dispositivo.
- Aislar endpoints comprometidos.
- Ejecutar acciones remotas

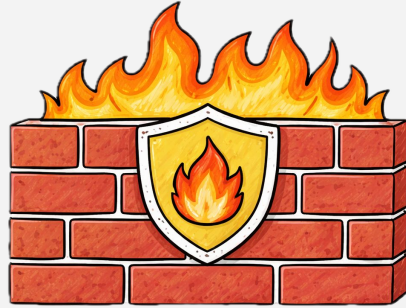
algunos ejemplos de herramientas EDR serian:

- CrowdStrike Falcon,
- Microsoft Defender para Endpoints,
- Sentinel One, o
- Carbon Black,

INTEGRACIÓN CON FIREWALLS Y CONTROLES DE RED

Queremos:

- Bloquear IPs o dominios sospechosos
- Crear reglas temporales
- Ajustar políticas de acceso
- Redirigir tráfico a análisis



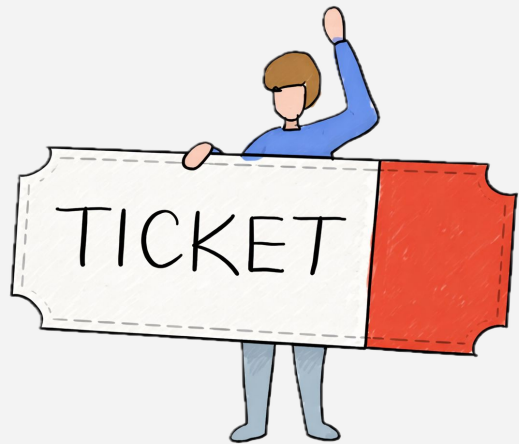
de la misma manera, podemos integrarnos con controles de la red para

- Bloquear IPs o dominios sospechosos
- Crear reglas temporales
- Ajustar políticas de acceso
- Redirigir tráfico a análisis

INTEGRACIÓN CON TICKETING

Queremos:

- Crear tickets automáticamente
- Actualizar estados de incidentes
- Adjuntar evidencias
- Escalar o cerrar casos



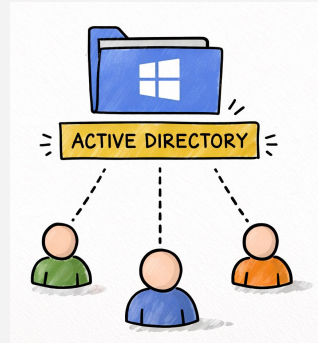
o integramos con apps de ticketing cómo Jira y

- Crear tickets automáticamente
- Actualizar estados de incidentes
- Adjuntar evidencias
- Escalar o cerrar casos

INTEGRACIÓN CON ACTIVE DIRECTORY

Queremos:

- Consultar datos del usuario
- Resetear contraseñas
- Deshabilitar cuentas
- Revocar accesos



o podemos integrarnos con el active directory para:

- Consultar datos del usuario
- Resetear contraseñas
- Deshabilitar cuentas
- Revocar accesos....

SCORING DE RIESGO



OK entonces automatizamos. Pero hay una pregunta previa: ¿cómo sabe el sistema qué alertas merecen atención primero? Para priorizar, tenemos el scoring de riesgo.

SCORING DE RIESGO

- Muchas alertas por día
- Tenemos que priorizar

Queremos:

- Asignar un número que indique qué tan probable es que una alerta sea una amenaza real.

3 DECISIÓN

```
Alto → caso HIGH.  
Medio → pausa para humano.  
Bajo → resuelve  
automáticamente.
```

Vimos que un SIEM puede generar miles de alertas por día. Los analistas no pueden investigarlas todas

Entonces necesitamos una forma de ordenarlas.

El scoring ayuda a hacer eso... le asigna un número a cada alerta que representa qué tan probable es que sea una amenaza real.

Es una puntuación que nos permite ordenar la cola de trabajo. Las alertas con score alto las investigamos primero. Las de score bajo las resolvemos automáticamente o las dejamos para el final, cuando ya resolvimos las de scores más altos."

¿Qué tener en cuenta para el scoring?

Alertas

- Tipo de eventos
- Severidad
- IOCs presentes

Usuarios

- Rol
- Historial de alertas
- Comportamiento típico

El activo

Servidor de producción vs. equipos de pruebas

Contexto

- Hora
- Ubicación geográfica
- Datos accedidos

Threat Intel

- IP en blocklist
- Patrón coincide con ataque conocido

| "¿De dónde viene ese número? De combinar múltiples señales. Ninguna señal sola es suficiente — una IP en blocklist puede ser falso positivo, un acceso a las 3 AM puede ser un analista trabajando tarde. Lo que importa es el conjunto.

Qué tenemos en cuenta para calcular el scoring?

| La alerta en sí: qué tipo de evento es, qué severidad tiene, si hay IOCs conocidos.

| El usuario: no es lo mismo que un admin con historial de alertas esta semana acceda a algo, que un usuario normal sin antecedentes.

| El activo: comprometer un servidor de producción financiero es muy distinto a comprometer un equipo de pruebas.

| El contexto: hora del día, desde dónde se conecta, a qué datos accede.

| Y threat intel: si la IP de origen ya está en una lista negra conocida

Cada categoría aporta puntos. Los sumás. El total es el score. Y ese número es lo que el playbook consume para decidir qué hacer

Veamos un ejemplo

Runbook vs. Playbook en [Google Colab](#) / [github](#)

Shuffle + alerta demo en [github](#)

78

Veamos el notebook..... [ABRIR COLAB]

primero tenemos el runbook, que es el procedimiento como documento, un conjunto de instrucciones para el analista. En este ejemplo vemos pasos con que tiene que hacer

Leer exactamente el código:

```
RUNBOOK_FUERZA_BRUTA = ""
```

```
RUNBOOK: Responder a Ataque de Fuerza Bruta SSH
```

```
=====
```

```
PASO 1 — Recolectar información del incidente
```

```
- Registrar la IP de origen
```

```
"""
```

Abajo vemos el mismo procedimiento pero en código, algo que podemos ejecutar

están los mismos pasos, pero bueno, para simplificar la demo ... esto es solo una simulación, por ejemplo no hay un chequeo real a CMDB...

leer celda por arriba...

también, puedo tener varias celdas e ir ejecutando una por una, y que el analista vea el resultado y según eso puede ir corriendo código...

van a ser **nodos visuales** en un workflow.

[VOLVER al slide , abrir github]

aca estaria el dockerfile y un README con el paso a paso de cómo levantarlo, pero bueno, una vez que lo tengamos hosteado localmente ...

abrimos Shuffle... `localhost:3001`

podemos ver algunos workflows de la comunidad

y aca tengo un workflow qué simula estos mismos 5 pasos...

Veamos por un lado:

- los 5 pasos - noten que son todos “reply to me” que es en escencia un print solo me da la informacon que le digo que me de... evidentemente en producción vamos a querer tener cosas funcionando, recolectando datos o corriendo scripts reales o conectandose a herramientas, para la demo, la deje solamente con un print
- las condiciones
- el webhook - qué poemas ir arrastrando elementos de la izquierda

en el webhook me copio la url y le doy start y save

ahora en la terminal voy a la carpeta demo y tengo un script de python que trigerea una alerta `cd ../demo`

Si corro el script con dry run, voy a ver el payload qué se va a mandar... `python trigger_alert.py --dry-run`

si yo le pongo el --URL manda una llamada al webhook... `python trigger_alert.py --url http://localhost:3001/api/v1/hooks/webhook\_05d19fc7-1eca-41ba-9af8-bc5e9f475f1a`

ahora nos vamos a la personita y podemos ver los runs... y vemos que fue por todo el workflow

si yo corro ahora agregando --ip 10.0.0.1, vemos que va al paso 1, y al 2 y después entra en la condicion de comenzar por 10. entonces deciamos falso positivo porque es interna y cerramos el caso

VOLVER A LA SIGUIENTE DIAPOSITIVA

“

Los runbooks nos dan estructura y los playbooks nos permiten ejecutar esa estructura.

Y el SOAR nos permite conectar herramientas y automatizar tareas.

78

bueno... si bien esta demo fue corta, me interesa que se queden con los conceptos de runbook, playbook, tipos de soar qué hay en el mercado y un ejemplo de un soar opensource autohosteado ejecutando un workflow... porque después en un escenario real, la idea es poder ir haciendo automatizaciones acorde al caso de uso... y según las políticas de cada empresa y las herramientas disponibles.

Lo importante a recordar es que

Los runbooks nos dan estructura y proceso, y los playbooks nos permiten ejecutar esa estructura

y un SOAR conecta herramientas, automatiza tareas y organiza la respuesta, pero lo realmente importante es cómo está diseñado ese proceso por dentro.

Muchas gracias por llegar hasta aca y qué sigan disfrutando de la automatización !

INTEGRACIÓN DE CONECTORES

Hola a todos! Soy Flor, bienvenidos de vuelta a la Especialización en seguridad defensiva
y hoy vamos a estar viendo integración de conectores

¿CON QUÉ HERRAMIENTAS SE CONECTA UN SOAR?

Categoría	Herramientas típicas	¿Qué hace el SOAR?
EDR	CrowdStrike, Defender	Aislar endpoints, ejecutar comandos
Firewall	Palo Alto, Fortinet	Bloquear IPs y dominios
Ticketing	Jira, ServiceNow	Crear y actualizar incidentes
Threat Intel	VirusTotal, MISP	Enriquecer IOCs con contexto
Email	Exchange, Proofpoint	Poner en cuarentena phishing
Identidades	Active Directory	Deshabilitar usuarios comprometidos

80

Antes de hablar del cómo, veamos el qué. El valor real del SOAR aparece cuando conectas las herramientas que ya usas en el SOC.

Por un lado, herramientas de detección y respuesta:

- el EDR que ve lo que pasa en los endpoints,
- el firewall que controla el tráfico.

Por otro, herramientas de enriquecimiento:

- VirusTotal para saber si un hash es malicioso,
- MISP para inteligencia de amenazas.

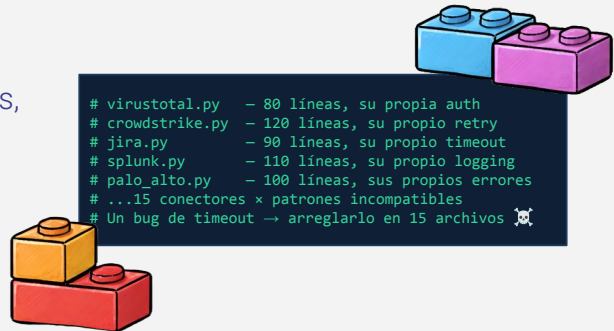
Y también tenemos lo administrativo:

- Jira para dejar registro de cada incidente,
- Active Directory para deshabilitar usuarios comprometidos

Cada categoría tiene una forma diferente de autenticarse, de hablar con su API, de devolver datos. Y ahí es donde empieza el problema.

EL PROBLEMA DEL CONECTOR AD-HOC

- Cada conector resuelve lo mismo de forma distinta
- Se repite autenticación, errores, timeouts y logging
- Los conectores se vuelven inconsistentes
- Mantenerlos escala mal



Veníamos viendo integraciones en scripts pero Si cada integración se hace como un script aislado, al principio parece simple:

- Escribís el conector de VirusTotal, funciona.
- Escribís el de CrowdStrike, funciona.

- Pero cuando llegás al

décimo conector, descubrís un bug en cómo manejas los timeouts. Y ese bug existe en los diez archivos, cada uno escrito de manera diferente. Tenés que arreglarlo en diez lugares distintos. Esto es

insostenible en producción ni cuando los playbooks escalen.

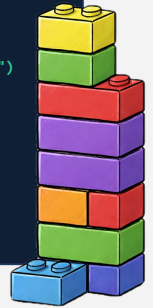
LA SOLUCIÓN: UNA INTERFAZ COMÚN

- Centralizamos las operaciones comunes
 - Autenticación
 - Manejo de errores
 - Logging
 - Formato estándar de respuesta
- Lo específico queda dentro de cada integración
- Agregar nuevos conectores es más simple

```
class ConectorBase:
    def ejecutar(self, accion):
        print("logging / validación / errores")
        return self._ejecutar(accion)

    def _ejecutar(self, accion):
        raise NotImplementedError

class ConectorEDR(ConectorBase):
    def _ejecutar(self, accion):
        return f"EDR: {accion}"
```



La solución: es tener **una interfaz abstracta común** que todos los conectores implementan.

Supongamos que la llamamos ConectorBase...

En ConectorBase viven todas las operaciones comunes:

- reintentos
- logging, y
- manejo de excepciones, y tenemos un
- formato uniforme de respuesta.

Los conectores concretos heredan todo eso y solo implementan lo que los diferencia: sus endpoints, su autenticación, su lógica propia.

LA SOLUCIÓN: UNA INTERFAZ COMÚN

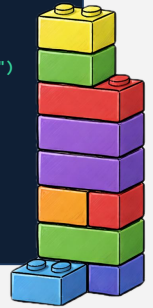
Cada conector implementa:

- Cómo hablar con su API
- Qué acciones soporta
- Cómo interpreta las respuestas
- Cómo traduce resultados al formato común

```
class ConectorBase:
    def ejecutar(self, accion):
        print("logging / validación / errores")
        return self._ejecutar(accion)

    def _ejecutar(self, accion):
        raise NotImplementedError

class ConectorEDR(ConectorBase):
    def _ejecutar(self, accion):
        return f"EDR: {accion}"
```



83

cada conector implementaria

- Cómo hablar con su API
- Qué acciones soporta
- Cómo interpreta las respuestas
- Cómo traduce resultados al formato común

Cada conector concreto solo debería encargarse de su parte específica: sus endpoints, sus credenciales y la lógica propia de la herramienta.

La idea es que Lo específico queda dentro de cada integración y Agregar nuevos conectores es más simple

Veamos un ejemplo

Demo de conectores en [Google Colab](#) / [github](#)

Código separado en conectores en [github](#)

86

Hasta ahora vimos cómo conectarse a una API puntual. Pero en un SOC no hablás con una sola herramienta — tenés VirusTotal, CrowdStrike, Jira, y decenas más. La pregunta es: ¿cómo organizás ese código para que no sea un caos?

La respuesta es este patrón: un framework de conectores. Miremos la estructura.

```
ejemplo-en-modulos-de-python/  
├── core/  
│   ├── base.py    ← contrato base  
│   └── registry.py ← registro central  
├── connectors/  
│   ├── virustotal.py  
│   ├── crowdstrike.py  
│   └── jira.py  
└── main.py        ← pipeline de respuesta
```

"Dos carpetas. core define las reglas del juego. connectors tiene los jugadores."

[Abrir core/base.py]

| "Empezamos por base.py. Esto es una clase abstracta — básicamente un contrato. Todo conector que yo agregue al framework tiene que implementar verificar_salud. Si no lo hace, Python directamente no me deja instanciarlo."

| "También tiene dos utilidades que todos heredan: logging con el nombre del conector, y reintentos con backoff exponencial. Si la API falla, reintentará tres veces antes de tirar error. Eso no lo tenés que escribir en cada conector."

[Abrir core/registry.py]

| "El registry Es el directorio de todos los conectores activos. Registrás una vez, y desde cualquier parte del código pedís el conector por nombre. main.py no importa VirusTotal directamente — le pregunta al registry. Eso desacopla la lógica de los conectores."

[Abrir connectors/virustotal.py]

| "Acá viene lo interesante. Cada conector tiene un flag use_mock. ¿Por qué? Porque cuando estás desarrollando o haciendo una demo, no querés consumir créditos de API ni necesitar credenciales reales.

El mock devuelve datos realistas — un hash de EICAR con 58 detecciones o... una IP maliciosa

[Abrir [main.py](#)]

| "Cuando estás listo para producción, solo cambiás USE MOCK = False en main.py y ponés las variables de entorno. Cero cambios en la lógica."

| "En main tenemos el pipeline de respuesta en la función responder_a_malware. Llegó una alerta El flujo es:"

| "Uno — enriquecimiento: le pasamos el hash a VirusTotal. Si no es malicioso, terminamos. No hacemos nada más y si si.. .continúa al paso siguiente."

| "Dos — localización: buscamos el endpoint en CrowdStrike."

| "Tres — contención: aislamos el dispositivo automáticamente, sin intervención humana."

| "Cuatro — documentación: creamos el ticket en Jira con todo el contexto ya cargado."

| "En producción, eso serían cuatro llamadas API en secuencia. Acá lo vemos con mock, pero la lógica sería la misma."

[Ejecutar python main.py en terminal]

| "Veamos cómo corre."

(dejar que corra y que se lea el output paso a paso)

| "Un endpoint aislado y un ticket abierto en menos de un segundo. En un SOC real esto se dispararía en respuesta a una alerta de SIEM — sin que un analista toque nada."

[Cierre]

RESUMEN

- Sin patron: 15 scripts ad-hoc, un bug = 15 fixes
- Con una clase base (ConectorBase): un fix, todos los conectores se benefician
- Registry: el código no sabe cómo se inicializó nada, solo pide lo que necesita.

Para cerrar:

Lo que vimos hoy es el patrón base que usan plataformas SOAR comerciales como Splunk SOAR o Palo Alto XSOAR por dentro.

Agregar un conector nuevo es copiar uno existente, heredar de Conector Base, y registrarlo. El resto del sistema no cambia.

la arquitectura de conectores es lo que determina si podemos crecer de 5 a 50 integraciones sin volverse inmantenible.

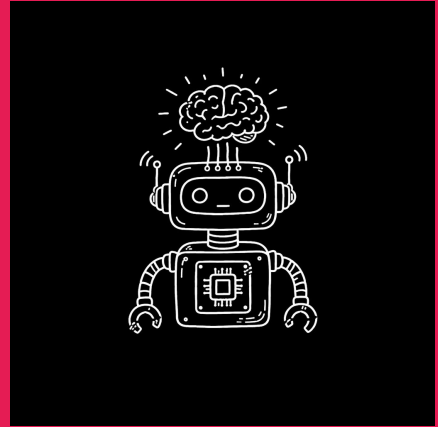
La clase base centraliza lo que siempre es igual.

El registry centraliza dónde viven.

Y cada conector concreto solo hace lo que lo hace único...

Bueno , eso sería todo por hoy.. nos vemos en el proximo video y espero que sigan disfrutando de la automatizacion !

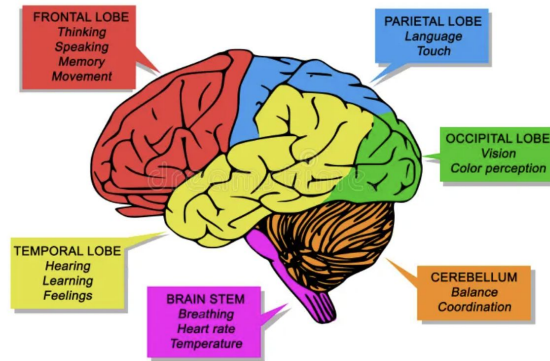
INTRODUCCIÓN A IA Y A MACHINE LEARNING



Hola a todos! soy Flor y bienvenidos de vuelta al curso de Seguridad Defensiva
hoy vamos a ver una introducción a IA y a Machine Learning

QUÉ ES LA IA?

El campo que engloba cualquier programa que intente replicar aspectos de la inteligencia humana...



87

Bien...

Ultimamente cuando se habla de inteligencia artificial, las personas en realidad están hablando de LLMs: modelos de lenguaje... pero

La **inteligencia artificial** es un campo amplio y va más allá que las LLMs...

La idea de la IA es hacer sistemas capaces de hacer tareas que normalmente asociamos con inteligencia humana. Una forma simple de pensarlo es compararlo con nuestras capacidades y nuestros **sentidos**

Por ejemplo, para la capacidad de visión, hay IAs que intentan “ver”, y ahí entra **computer vision**, que analiza imágenes y video.

para la audición, hay los modelos que reconocen voz, sonidos o música.

para la **coordinación motora**, el equilibrio y el control del cuerpo. hay IAs que buscan “moverse” o coordinar acciones en el mundo físico, como en robótica.

También hay modelos que trabajan con “el lenguaje”, que sería lo más parecido a nuestra capacidad de hablar, leer y entender texto. Ahí entran los **LLMs**

Entonces, en vez de pensar la IA como una sola cosa, conviene verla como un conjunto de tecnologías que intentan replicar distintas capacidades humanas: ver, escuchar, hablar, decidir o actuar.

QUÉ ES LA IA?

El campo que engloba cualquier programa que intente replicar aspectos de la inteligencia humana...

Incluyendo programas basados en reglas (sistemas expertos).

Ej: sistema de diagnóstico de enfermedades



Reglas del sistema	
SI: fiebre + tos + dolor de cuerpo + fatiga	→ Gripe (Influenza)
SI: congestión + estornudos + dolor de garganta	→ Resfriado común
SI: fiebre + dolor de garganta	→ Posible faringitis
SI: tos + fatiga	→ Posible bronquitis
SI: congestión + estornudos (sin fiebre)	→ Posible alergia



El ejemplo más clásico de IA antes del machine learning eran los sistemas expertos.

por ejemplo un sistema de diagnóstico de enfermedades donde la entrada es una lista de síntomas y la salida es un diagnóstico.

Un médico le decía a un programador "si el paciente tiene fiebre, tos y dolor de garganta, probablemente es gripe".

El programador escribía eso como reglas: IF fiebre AND tos AND dolor de garganta THEN gripe. Funcionaba para problemas acotados.

El problema es que para programar todo el conocimiento de un médico real, necesitabas escribir miles de reglas a mano. Y si cambiaba algo cómo por ejemplo una nueva enfermedad o un nuevo síntoma, alguien tenía que entrar a cambiar el código.

PROGRAMAS ESCRITOS CON REGLAS

Antes del machine learning, todos los programas de detección tenían que ser programados explícitamente con reglas, línea por línea: "si esto entonces aquello"



89

todos los programas de detección tenían que ser programados explícitamente con reglas, línea por línea: "si esto entonces aquello"

para seguridad específicamente, ese enfoque no servía por dos razones:

1. las amenazas son complejas y difíciles de traducir a reglas exhaustivas, y
2. el comportamiento cambia. En el momento en que se define una regla, el atacante ya encontró una manera de evadirla.

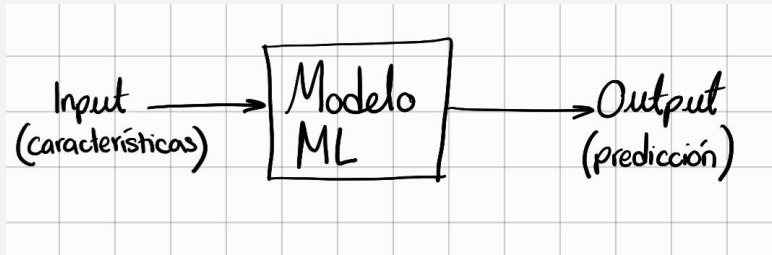
y por suerte apareció el Machine Learning, que es una rama de la inteligencia artificial donde los algoritmos **aprenden patrones de datos** en vez de ser programados explícitamente con reglas.

En vez de escribir cada condición, le das ejemplos al algoritmo y el algoritmo infiere automáticamente qué patrones son relevantes.

Vendría a ser un cambio de paradigma

Modelo de Machine Learning

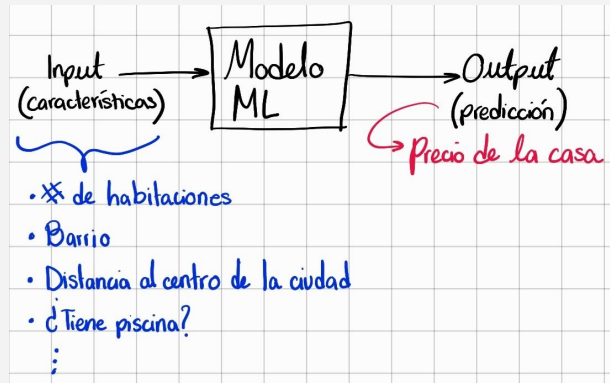
El objetivo es construir un modelo que pueda predecir algo (output) basándose en datos de entrada (input)



En ML, el objetivo es construir un **modelo** que pueda **predecir** algo basándose en datos de entrada.

Modelo de Machine Learning

Ejemplo: modelo para predecir el precio de una casa.



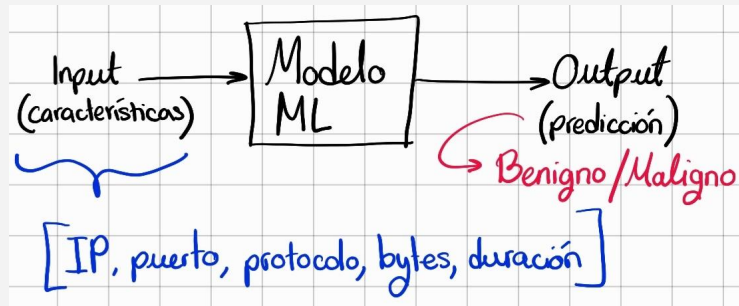
Por ejemplo, un modelo puede predecir el precio de una casa.

La salida (output) es el precio;

la entrada (input) son las características de la casa: cantidad de habitaciones, barrio, distancia al centro, si tiene jardín, cuantos metros cuadrados tiene, etc.

Modelo de Machine Learning

Ejemplo: modelo para predecir si un evento en el tráfico de red es tráfico normal o un posible ataque.



El mismo concepto lo podemos aplicar a temas de seguridad: por ejemplo predecir si un evento en el tráfico de red es tráfico normal o un posible ataque.

aca, las características de entrada ya no serian los metros cuadrados de una casa, serian características del evento, por ejemplo: IPs, el puerto, protocolo, bytes, o la duración.

APRENDIZAJE SUPERVISADO VS. NO SUPERVISADO



93

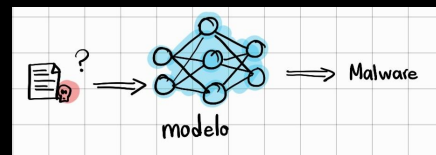
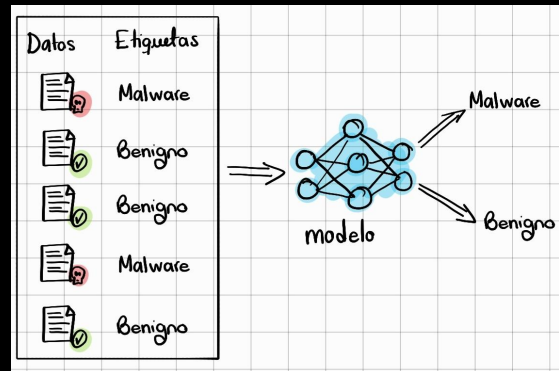
OK entonces queremos entrenar un modelo... pero primero hay que entender que hay dos grandes paradigmas, y vamos a usar los dos por razones distintas. Estos paradigmas son:

- aprendizaje supervisado
- aprendizaje no supervisado

Aprendizaje Supervisado

El dataset de entrenamiento contiene ejemplos **etiquetados** (con respuestas correctas ya conocidas) de lo que queremos detectar.

El modelo aprende cuáles características (**features**) tienen más peso para hacer una predicción correcta.



En el aprendizaje supervisado, el dataset de entrenamiento tiene etiquetas claras. Hablamos de qué ya tenemos ejemplos con respuestas correctas ya conocidas

Por ejemplo podemos tener un dataset donde ya se sabe si un archivo es malware o no. o en el ejemplo de la casa... la respuesta que queremos conocer va a ser el precio, o sea puede ser un valor numérico.

El modelo aprende a asociar las características con la etiqueta correcta. Al final del entrenamiento, cuando le mostrás un ejemplo nuevo sin etiqueta, puede predecir la etiqueta y por ejemplo en un modelo qué aprendió a predecir malware puede decirte si es malware o no basándose en lo que aprendió.

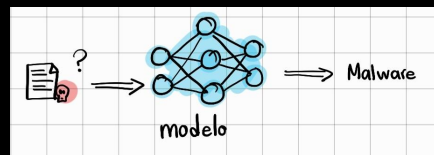
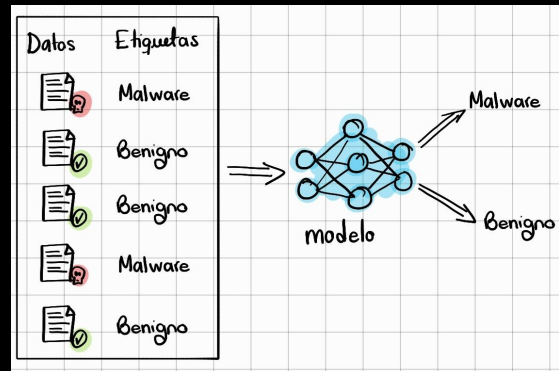
Aprendizaje Supervisado

Ventajas

- Efectivo si las etiquetas son correctas

Desventajas

- Requiere etiquetado manual (costoso)
- No detecta variantes nunca antes vistas



Estos modelos son muy efectivos si tenemos las etiquetas correctas

pero requiere un etiquetado manual y esto es costoso: alguien tiene que revisar miles o millones de ejemplos y decir "este es malware, este no".

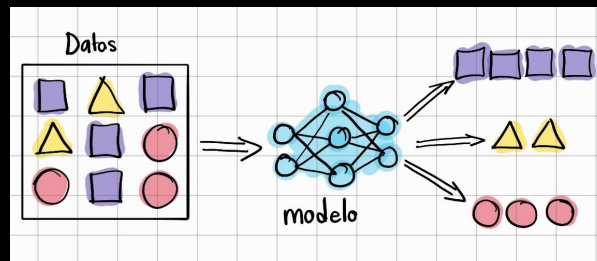
Eso es caro y lento.

y también otra desventaja de estos modelos es que no van a poder detectar variantes que nunca vimos antes.

Para amenazas conocidas, supervisado. Para amenazas nuevas, necesitamos el otro paradigma.

Aprendizaje No Supervisado

- El dataset de entrenamiento **no tiene etiquetas**
- El modelo intenta encontrar patrones ocultos en los datos: busca similitudes y las agrupa.
- Cuando un dato nuevo llega, indica a qué grupo pertenece.



En el aprendizaje no supervisado, no hay etiquetas. El modelo no sabe qué es malware y qué es benigno — no tiene esa información. Lo que hace es buscar patrones ocultos:

agrupa datos similares entre sí, y cuando llega un dato nuevo, lo ubica dentro de uno de esos grupos o lo marca como algo que no encaja en ninguno.

Eso último — lo que no encaja en ningún grupo — es exactamente lo que nos interesa en detección de anomalías.

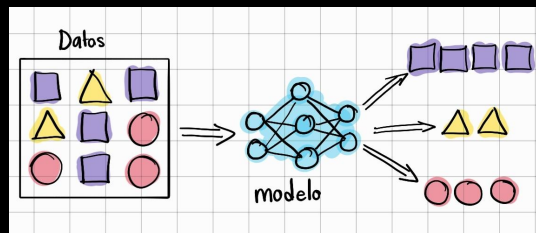
Aprendizaje No Supervisado

Ventajas:

- No requiere etiquetado manual
- Puede **detectar anomalías** o patrones inesperados

Desventajas:

- Es difícil validar si el resultado es correcto
- Los clusters pueden no corresponder a "malware vs benigno"



97

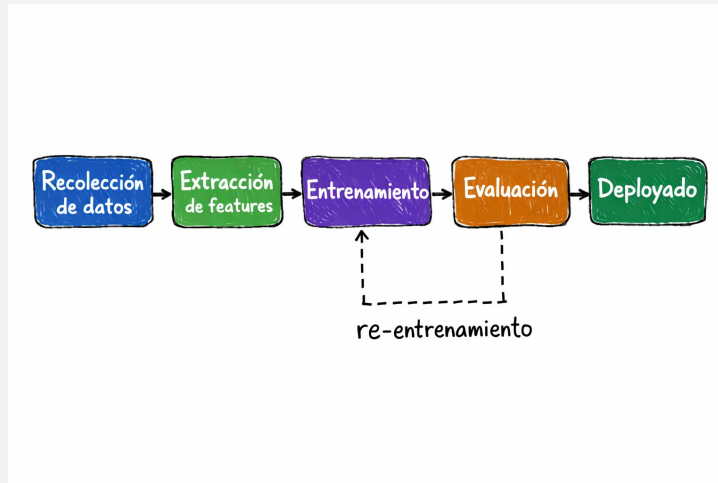
La ventaja es que no requiere etiquetado manual y puede detectar patrones inesperados que nadie anticipó. por ejemplo un ataque completamente nuevo, que nunca existió antes, puede aparecer como una anomalía

La desventaja es la validación: ¿cómo sabés si lo que el modelo agrupó corresponde a algo significativo? Las agrupaciones pueden no corresponder a "malware vs benigno" — pueden

ser qué estén agrupados por horarios, sistemas operativos, tipos de usuarios.

Interpretar los resultados

Workflow para armar un modelo de Machine Learning



98

Veamos el proceso para armar un modelo.

Arranca con la recolección de datos — logs, alertas, netflow, lo que tengas.

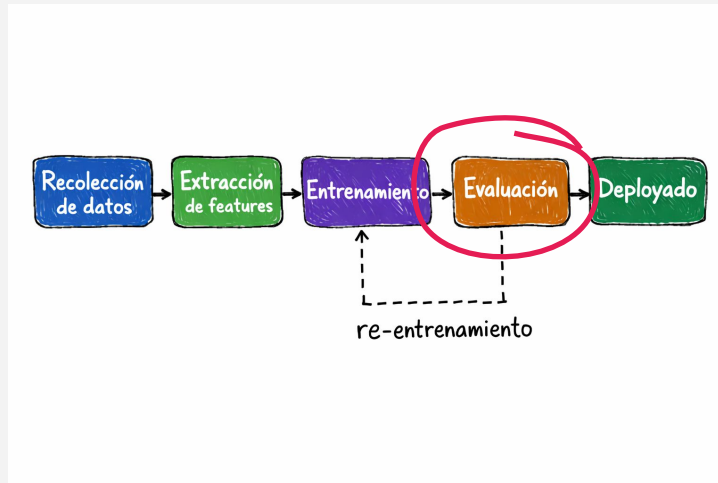
Después viene la extracción de features: transformar esos datos crudos en las características que el modelo va a usar.

Después el entrenamiento y

la evaluación... que en esta fase probamos el modelo en datos nuevos, que nunca vio antes, para poder medir su rendimiento en la vida real. Estos datos nuevos se llaman test set.

Si el modelo se desempeña bien, lo deployamos a producción.

Workflow para armar un modelo de Machine Learning



entrenar modelos es fácil....

costoso en términos computacionales, pero fácil

lo complicado después es entender cuál modelo es útil para usar en tu sistema y para eso tenemos que evaluar los modelos y poder tomar decisiones.

las evaluaciones las hacemos tomando en cuenta diferentes métricas

Métricas de evaluación

Accuracy

Predicciones correctas / total.

Precision

De los que dije 'malware', ¿cuántos lo son realmente? Reduce falsos positivos.

Recall

De todo el malware real, ¿cuánto detecté? Reduce falsos negativos.

- TP: Verdaderos Positivos
- TN: Verdaderos Negativos
- FP: Falsos Positivos
- FN: Falsos Negativos

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Accuracy es lo más básico: de todas las predicciones que hizo el modelo, ¿cuántas fueron correctas? El problema es que en seguridad los datasets son muy desbalanceados — si el 99% de los eventos son

benignos y el modelo dice "benigno" para todo, tiene 99% de accuracy y no sirve para nada.

Por eso necesitamos Precision y Recall, que miran el problema desde ángulos distintos.

Precision te dice.. de todo lo que el modelo marcó como malware, ¿cuánto era realmente malware? Un modelo con baja precision genera muchas falsas alarmas

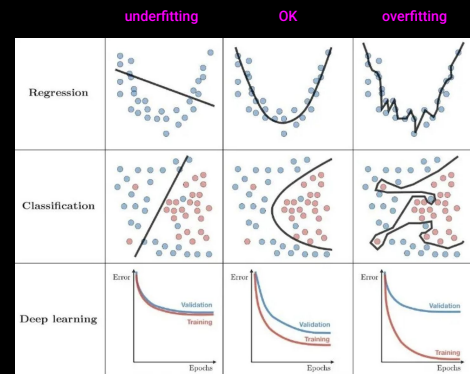
Recall te dice de todo el malware real que existía, ¿cuánto detectó el modelo? Un modelo con bajo recall deja amenazas pasar sin detectarlas.

Subir el recall generalmente baja la precision, y viceversa. Si querés detectar todo, vas a generar más falsas alarmas. Si querés pocas falsas alarmas, vas a dejar pasar más amenazas.

Pero por más que querramos un recall alto, también queremos razonable **precision** (no demasiados falsos positivos que abrumen a los analistas).

¿Por qué no validamos sobre el conjunto de entrenamiento? ... OVERFITTING

	Subajuste (underfitting)	OK	Sobreajuste (overfitting)
	El modelo es muy simple	El modelo tiene la complejidad adecuada	El modelo es demasiado complejo
Síntomas	- Error de entrenamiento alto - Error de entrenamiento cercano al error de prueba - Alto sesgo	El error de entrenamiento es ligeramente menor que el de prueba	- Bajo error de entrenamiento - El error de entrenamiento es mucho menor que el error de prueba - Alta varianza
Cómo resolver	- Complejizar el modelo - Agregar más características (features) - Entrenar por más tiempo - Obtener más datos	-	- Simplificar el modelo - Regularizar el modelo (L1, L2, Dropout) - User early stopping - Obtener más datos



¿Por qué no evaluamos el modelo con los mismos datos que usamos para entrenarlo?

Porque si lo hacés, el modelo va a tener performance perfecta usamos los datos de entrenamiento y el modelo aprendió esos datos de memoria. Eso se llama overfitting. Un modelo que memorizó el training set aprendió el ruido específico de esos datos, no el patrón real.

Cuando llegan datos nuevos, falla.

La solución es siempre evaluar con datos que el modelo nunca vio durante el entrenamiento.

El underfitting es el otro extremo: el modelo es demasiado simple para capturar el patrón.

El objetivo es el punto medio, buscamos que el modelo logre una generalización.

Ajuste continuo del modelo

- Periódicamente (mensualmente o trimestralmente), **reunís nuevos datos** que fueron etiquetados por analistas y **reentrenás** el modelo.
- Esto asegura que el modelo se adapte a nuevas amenazas y características de tu entorno específico.

```
Caso #123:  
  Predicción: Score 85 (CRITICAL)  
  Veredicto analista: Falso positivo (fue error de configuración)  
  → Modelo fue incorrecto, agregar caso en el dataset para re-entrenar  
  
Caso #456:  
  Predicción: Score 25 (LOW)  
  Veredicto analista: Verdadero ataque (después de investigación)  
  → Modelo falló, agregar caso en el dataset para re-entrenar
```

Un modelo no es algo que entrenás una vez y olvidás. Las amenazas evolucionan, tu infraestructura cambia, los patrones de comportamiento de los usuarios cambian.

Periódicamente — mensualmente o

trimestralmente — reunís datos nuevos que fueron etiquetados por los analistas durante la operación.

Y vamos armando un dataset de re-entrenamiento.

Así el modelo se adapta a tu entorno específico y a las amenazas nuevas que van apareciendo.

Limitaciones del ML en seguridad

Cold Start Problem

No se puede entrenar para detectar amenazas nuevas sin ejemplos previos.

Ataques Adversariales

El atacante puede manipular su malware para evadir el modelo al conocer qué características detecta.

Garbage In, Garbage Out

Datos de entrenamiento incorrectos o sesgados producen un modelo incorrecto o sesgado.

Falta de Explicabilidad

Las redes neuronales son 'cajas negras': dicen 'malware' sin explicar por qué. Los analistas necesitan el razonamiento.

Concept Drift

Si el malware evoluciona (y siempre evoluciona), el modelo entrenado en 2020 falla en 2024.

Alto Costo de Falsos Positivos

1% de FP sobre millones de archivos = cientos de alertas falsas diarias → analistas abrumados.

Solución: combinar ML + reglas de negocio + revisión humana (human-in-the-loop) + monitoreo de concept drift

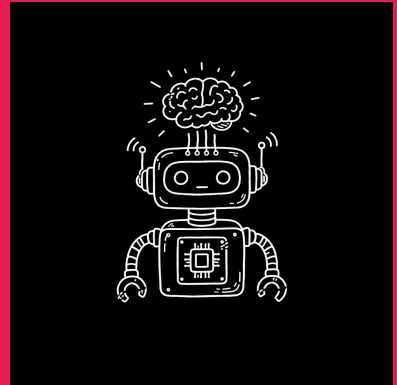
103

Pero incluso el ML tiene limitaciones:

- El Cold Start Problem: es que no podés entrenar un modelo para detectar amenazas si no tenés ejemplos previos. Alguien tiene que haber visto ese ataque antes para que el modelo lo aprenda.
- Los Ataques Adversariales: un atacante que conoce tu modelo puede diseñar malware específicamente para evadirlo — pueden hacer pequeñas modificaciones en las características para que caiga del lado "benigno" de la frontera.
- Garbage In, Garbage Out: cuando los datos de entrenamiento incorrectos o sesgados producen un modelo incorrecto o sesgado.
- La Falta de Explicabilidad: las redes neuronales son cajas negras — el modelo dice "malware" pero no puede explicar por qué de una forma que un analista pueda verificar.
- El Concept Drift: el malware evoluciona y el modelo entrenado en 2020 falla en 2024.
- Y el Alto Costo de Falsos Positivos: en un SOC grande, 1% de Falsos Positivos sobre millones de eventos son decenas de miles de alertas

La solución no es reemplazar reglas con ML, sino combinarlos: reglas para lo conocido, ML para lo desconocido, revisión humana para lo ambiguo.

ML EN SEGURIDAD Y DETECCIÓN DE ANOMALÍAS



Hola a todos! soy Flor y bienvenidos de vuelta al curso de Seguridad Defensiva

hoy vamos a ver ML EN SEGURIDAD y detección de anomalías

Objetivos

- Conocer algoritmos comunes de ML y cuándo aplicarlos
- Entender qué es la detección de anomalías
- Comprender UEBA y cómo se integra con el SIEM

En esta sección vamos

- Conocer algoritmos comunes de ML y cuándo aplicarlos
- Entender qué es la detección de anomalías
- Comprender UEBA y cómo se integra con el SIEM

Algoritmos comunes en seguridad

Árboles

- Árbol de Decisión – reglas binarias, fácil de interpretar
- Random Forest – múltiples árboles, reduce sobreajuste

Modelos Lineales

- Regresión Logística – clasificación binaria simple
- SVM – frontera óptima que maximiza separación

Redes Neuronales

- DNN – múltiples capas, muy potentes
- Requieren grandes datos y tiempo de entrenamiento

Anomalías

- Isolation Forest – aísla comportamientos raros
- Autoencoders – aprenden a reconstruir lo normal
- One-Class SVM – entrena solo con datos normales

Clustering

- K-Means – grupos por similitud
- DBSCAN – densidad variable, outliers = sospechoso

Dimensionalidad

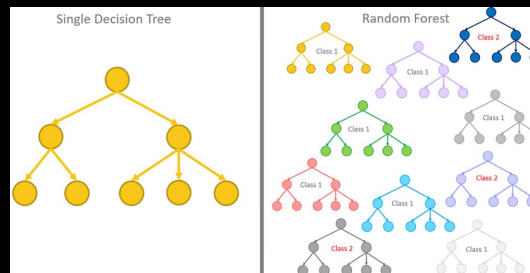
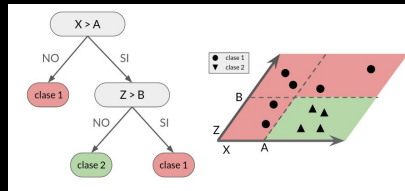
- PCA – reduce variables, mantiene información
- t-SNE / UMAP – visualizar clústers en 2D

No existe un único algoritmo que funcione para todo. Cada familia de modelos tiene un propósito distinto, y elegir bien depende de qué problema estás tratando de resolver.

Algoritmos comunes en seguridad

Árboles

- Árbol de Decisión - reglas binarias, fácil de interpretar
- Random Forest - múltiples árboles, reduce sobreajuste



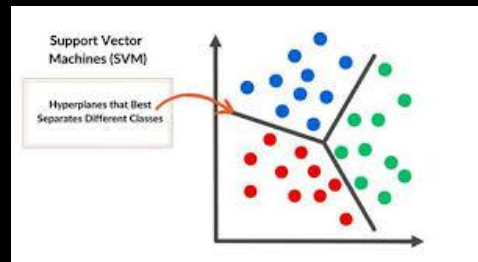
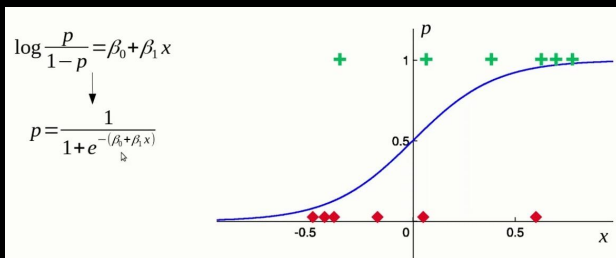
Arrancamos con los árboles.

El Árbol de Decisión toma decisiones binarias es el modelo más fácil de explicar a un manager o a un equipo legal porque podés mostrar exactamente por qué tomó una decisión, pero puede tener un overfitting.. entonces surgió Random Forest que agrega cientos de árboles entrenados con datos distintos y combina sus votos

Algoritmos comunes en seguridad

Modelos Lineales

- Regresión Logística - clasificación binaria simple
- SVM - frontera óptima que maximiza separación



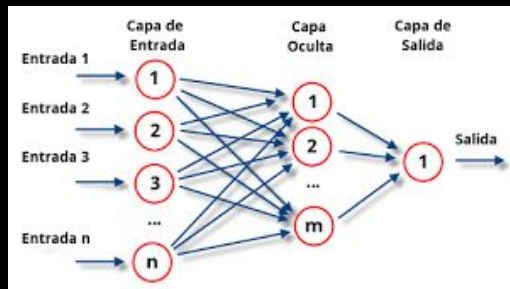
108

Después tenemos modelos lineales cómo:

Regresión Logística que te da una probabilidad — por ejemplo: "este email tiene un 94% de ser phishing" o como

SVM que busca la frontera óptima que maximiza la separación entre clases, y funciona muy bien cuando las clases están bien definidas pero el dataset no es enorme.

Algoritmos comunes en seguridad



Redes Neuronales

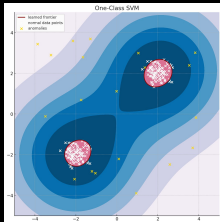
- DNN – múltiples capas, muy potentes
- Requieren grandes datos y tiempo de entrenamiento

Las redes neuronales son las más potentes, pero necesitas muchos datos y tiempo de entrenamiento.

las usas cuando los patrones son demasiado complejos para los otros modelos como

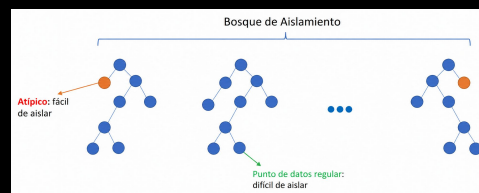
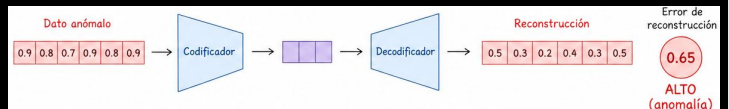
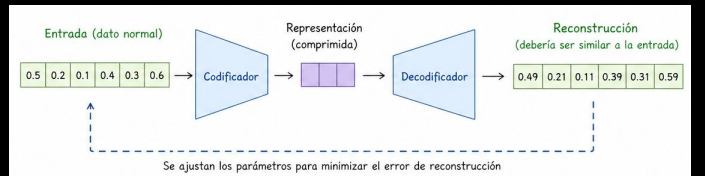
- análisis de comportamiento a escala,
- detección de malware por características de los archivos binarios,
- procesamiento de logs no estructurados.

Algoritmos comunes en seguridad



Anomalías

- Isolation Forest - aísla comportamientos raros
- Autoencoders - aprenden a reconstruir lo normal
- One-Class SVM - entrena solo con datos normales

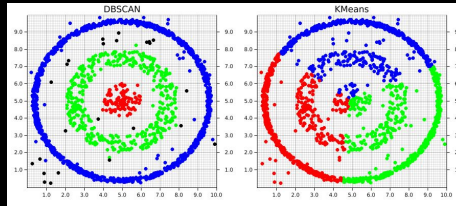
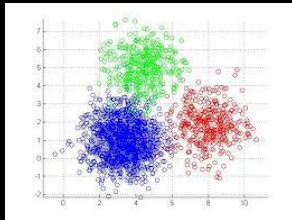


Modelos de Anomalías: estos modelos no aprenden qué es malicioso — aprenden qué es normal, y alertan sobre lo que se desvía.

- por ejemplo los Autoencoders aprenden a comprimir y reconstruir datos normales — si le mando un dato nuevo y no lo pueden reconstruir bien, es sospechoso.
- o One-Class SVM entrena solo con datos normales y marca como anómalo todo lo que queda fuera del límite.

Algoritmos comunes en seguridad

K-means

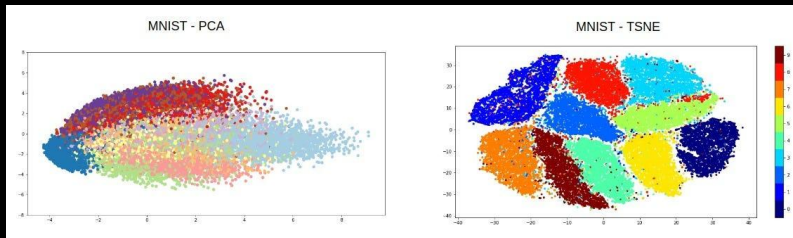
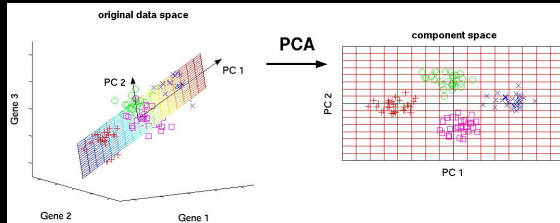


Clustering

- K-Means – grupos por similitud
- DBSCAN – densidad variable, outliers = sospechoso

Clustering agrupa comportamientos similares sin etiquetas previas. Estos algoritmos crean grupos llamados clusters y trata los puntos aislados como outliers, y estos outliers son lo que nos interesa

Algoritmos comunes en seguridad



Dimensionalidad

- PCA – reduce variables, mantiene información
- t-SNE / UMAP – visualizar clústers en 2D

Y modelos de reducción de dimensionalidad:

- reducen cientos de columnas para quedarnos con las características que nos dan más información, las más útiles.

Algoritmos comunes en seguridad

Árboles

- Árbol de Decisión – reglas binarias, fácil de interpretar
- Random Forest – múltiples árboles, reduce sobreajuste

Modelos Lineales

- Regresión Logística – clasificación binaria simple
- SVM – frontera óptima que maximiza separación

Redes Neuronales

- DNN – múltiples capas, muy potentes
- Requieren grandes datos y tiempo de entrenamiento

Anomalías

- Isolation Forest – aísla comportamientos raros
- Autoencoders – aprenden a reconstruir lo normal
- One-Class SVM – entrena solo con datos normales

Clustering

- K-Means – grupos por similitud
- DBSCAN – densidad variable, outliers = sospechoso

Dimensionalidad

- PCA – reduce variables, mantiene información
- t-SNE / UMAP – visualizar clústers en 2D

Ahora, las tres categorías de abajo son las que más vas a usar en detección de anomalías

Detección de anomalías



Arrancamos con detección de anomalías

ANOMALY DETECTION

Algo que se desvía de lo normal.

Las reglas no escalan para esto porque "lo normal" es diferente para cada usuario, cada sistema, cada horario...

Una anomalía es algo que se desvía de lo normal.

Las reglas no escalan para esto porque "lo normal" es diferente para cada usuario, cada sistema, cada horario.

Lo que el ML puede hacer es aprender ese "normal" de forma automática y personalizada, y detectar desviaciones sin que nadie tenga que definirlas a mano.

Análisis de Comportamiento de Usuarios (UBA)

UBA (User Behavior Analytics) analiza el comportamiento de los usuarios para detectar actividades sospechosas o inusuales.

UBA, User Behavior Analytics (o Análisis de Comportamiento de usuarios), es la aplicación de detección de anomalías al comportamiento de personas. En lugar de mirar el tráfico de red, miramos qué hace cada usuario específicamente: a qué hora se conecta, desde dónde, a qué sistemas accede, cuántos archivos descarga, con qué frecuencia. El modelo aprende el perfil de comportamiento normal de cada usuario y alerta cuando ese perfil cambia de forma significativa.

¿Qué patrones analiza UBA?

Volumen

Cuántos datos mueve el usuario.
Normal: 50 registros/día
Sospechoso: 50,000 registros/día de repente

Frecuencia

Con qué frecuencia realiza ciertas acciones.
Normal: 2-3 logins/día
Sospechoso: 50 logins en un día

Ubicación

Desde dónde se conecta el usuario.
Normal: oficina en Chicago
Sospechoso: acceso desde Beijing sin previo aviso

Peer Groups

Comparación con usuarios del mismo rol.
Si todos en el grupo hacen X pero este usuario hace Y → alerta.
Perfil fijo o dinámico.

Secuencias Anómalas

No solo QUÉ hace el usuario sino en qué ORDEN.
Ej: crear cuenta → usarla → borrarla → repetir. Ningún motivo legítimo.

UBA mira el comportamiento desde diferentes ángulos.

- Volumen: Juan normalmente mueve 50 registros por día — si de repente mueve 50.000, algo pasó.
- Frecuencia: si alguien hace 2-3 logins por día y de repente hace 50 en un día, es sospechoso.
- Ubicación: el usuario siempre se conecta desde la oficina en Estados Unidos — un acceso desde China sin previo aviso es una anomalía.
- Peer Groups: si todos en el equipo de finanzas hacen X y solo este usuario hace Y, hay que investigar
- Y Secuencias Anómalas: no solo qué hace el usuario sino en qué orden... Crear una cuenta, usarla para descargar datos, borrarla inmediatamente por ejemplo— esa secuencia no tiene ningún motivo legítimo.

UEBA (User and Entity Behavior Analytics)

Es la evolución de UBA.

Agrega la letra “E” de Entidades (Todo lo que no es una persona pero genera actividad en la red):

- Servidores
- Routers y switched
- BBDD
- Aplicaciones
- Dispositivos IoT

UEBA es la evolución de UBA. Agrega la letra E de Entidades — todo lo que no es una persona pero genera actividad en la red.

Servidores que de repente hacen conexiones salientes inusuales.

Routers con tráfico de control anómalo.

Bases de datos que reciben queries masivos fuera del horario normal.

Aplicaciones con patrones de acceso que cambiaron.

Dispositivos IoT que empezaron a comunicarse con IPs externas.

El modelo de comportamiento ya no es solo por usuario, sino por cada entidad de la red.

UEBA + SIEM

- Reducir ruido
- Priorizar investigaciones
- Escalar con el tamaño

Fuentes de datos → SIEM → UEBA → Lista de usuarios/entidades más riesgosos → Investigación

UEBA va integrado con el SIEM.

El flujo es: las fuentes de datos alimentan al SIEM, el SIEM alimenta al motor de UEBA, y el UEBA produce una lista priorizada de usuarios y entidades más riesgosos.

Ese output reduce el ruido porque en lugar de miles de alertas individuales tenés una lista corta de los casos que más importan; prioriza investigaciones para que el analista empiece por donde más impacto tiene; y escala con el tamaño de la organización sin necesitar más analistas proporcionalmente.

Importancia de UEBA en el SOC

Sin UEBA, un analista tendría que revisar manualmente la actividad de miles de usuarios cada día. Eso es imposible.

Con UEBA, el analista recibe algo así:

"Top 10 usuarios más riesgosos hoy:"

1. Juan – riesgo: crítico
2. ...

Ahora el analista sabe exactamente dónde enfocar su tiempo e investigación.

Sin UEBA, un analista tendría que revisar manualmente la actividad de miles de usuarios cada día. Eso es imposible — nadie tiene ese tiempo.

Con UEBA, el analista tendría una lista por ejemplo "Top 10 usuarios más riesgosos hoy." El analista no necesita buscar la anomalía. El sistema ya la encontró, la cuantificó, y la puso en

al comienzo de la lista. El analista ahora puede enfocarse en investigar a este usuario, entender el contexto, y decidir si es un incidente real.

Eso es lo que hace UEBA: convierte un problema de búsqueda en un problema de decisión.

[FINAL]



El ML permite detectar lo que las reglas no pueden: patrones ocultos, comportamientos anómalos y amenazas que nunca antes vimos.

El ML permite detectar lo que las reglas no pueden: patrones ocultos, comportamientos anómalos y amenazas que nunca antes vimos.

NLP Y LLMS EN SEGURIDAD

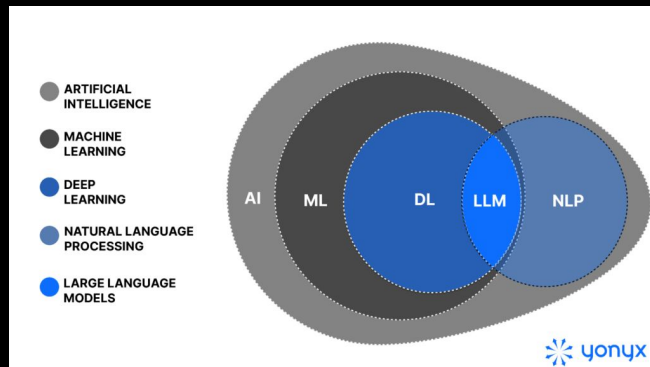
Hola a todos! soy Flor y bienvenidos de vuelta al curso de Seguridad Defensiva de Hackademy

Hoy vamos a ver NLP y LLMS en seguridad

¿QUÉ ES NLP?

Campo de la IA que permite a las máquinas entender y procesar lenguaje humano.

NLP $\neq$ LLMs



123

NLP (Natural Language Processing) es el campo de la IA que permite a las máquinas entender y procesar lenguaje humano.

Texto, básicamente.

NLP no es lo mismo que LLMs. Los LLMs son una forma de hacer NLP, pero NLP es el campo más amplio.

Con NLP podemos hacer clasificadores de texto, traductores automáticos y detectores de spam y esta tecnología existió mucho antes de que existiera GPT.

Por qué nos interesa aprender NLP para seguridad?

- En seguridad manejamos constantemente:
 - logs
 - alertas
 - tickets
 - reportes de malware
- NLP es el conjunto de técnicas para procesar ese texto:
 - clasificarlo
 - extraer información
 - detectar patrones

En seguridad manejamos texto constantemente: logs, alertas, tickets, reportes de malware.

NLP es el conjunto de técnicas para procesar ese texto automáticamente — clasificarlo, extraer información,

detectar patrones. Eso es lo que nos interesa.

Clasificación con NLP

Aplicar una etiqueta a un texto.

Útil cuando las categorías se pueden inferir del contenido textual en sí (las palabras nos indican qué categoría va a ser)

Ejemplos:

- Categorizar el mensaje de una alerta como fuerza bruta, exfiltración, reconocimiento
- Categorizar reportes de threat intel (ransomware, phishing, vulnerability, APT)
- Clasificar mails en spam/phishing/legítimo

Hay diferentes casos de uso para NLP, pero con NLP y machine learning podemos hacer modelos de clasificación en los que la entrada sea texto.

Clasificar significa asignar una etiqueta a un texto.

Clasificación con NLP funciona bien cuando el input es texto y las categorías se pueden inferir del contenido textual en sí — es decir, cuando las palabras que aparecen ya dicen a qué categoría pertenece el evento.

Casos de uso donde encaja en un SOC o en un equipo construyendo detecciones:

por ejemplo

- tenemos una alerta y por el mensaje podemos clasificar esta alerta como fuerza bruta, exfiltración, reconocimiento, etc
- podemos Categorizar reportes de threat intel (ransomware, phishing, vulnerability, APT)
- o Clasificar mails en spam/phishing/legítimo

Tokenización



Y un concepto necesario antes de continuar es la tokenización: convertir a tokens.

Antes de que una máquina pueda procesar texto, necesita dividirlo en partes. generalmente esas partes son palabras.. Eso es tokenizar

A partir de ahí el modelo trabaja con esas unidades, no con el texto crudo.

¿Cómo funciona un LLM?



127

ahora si... veamos cómo funcionan las LLMs

Porque si entendés cómo funciona , vas a entender por qué alucinan, cuándo confiar en los modelos y cuándo no, qué tipo de preguntas tienen sentido hacerles, y por qué nunca deberías poner datos sensibles de tu organización en un modelo que corre en la nube. Todo se deriva de ese entendimiento.



En 2017 un equipo de Google publicó un paper: "Attention is all you need"

Qué introdujo la arquitectura de IA "Transformer"

Todo empezó cuando un equipo de ocho investigadores de Google publicó en 2017 el paper titulado Attention Is All You Need. Ahí fue cuando introdujeron por primera vez la arquitectura de IA conocida como Transformer. Este fue un trabajo revolucionario y fue citado más de cien mil veces, lo cual es una barbaridad para un paper académico.

Arquitectura Transformer

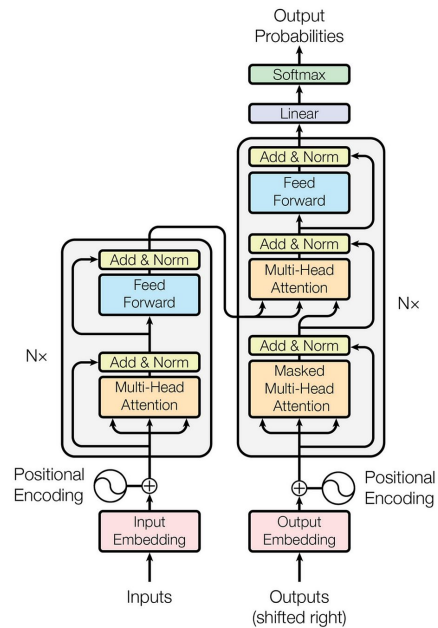


Figure 1: The Transformer - model architecture.

La arquitectura Transformer se ve más o menos así. No vamos a entrar en demasiado detalle porque es un tema bastante técnico, pero quedate con la idea de que tiene dos partes: un *encoder* y un *decoder*. Originalmente se diseñó para tareas de traducción automática, o sea pasar de un idioma a otro.

Después, algunos investigadores de OpenAI descubrieron que si agarrás esta arquitectura y eliminás el encoder, te queda un modelo basado solo en el decoder. Y resulta que el encoder es especialmente bueno generando texto.

Y justamente esa es la arquitectura que usan los modelos de lenguaje más potentes hoy en día, como GPT, LLaMA y Claude.

los modelos de lenguaje grandes nacieron como una versión “solo decoder” de los Transformers.

¿Qué necesitamos para hacer un LLM?

Muchos datos



Para tener un LLM necesitamos los siguientes elementos:

- Primero, necesitamos muchísimos datos. Estos datos los tenemos recorriendo internet con programas llamados “crawlers”, que van juntando información de todos lados.
 - Recolectan páginas web, Wikipedia, libros y mucho texto en general.. En resumen, necesitamos una cantidad enorme de información para poder funcionar.

¿Qué necesitamos para hacer un LLM?

Muchos datos



Arquitectura
Transformer

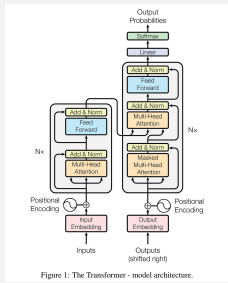


Figure 1: The Transformer - model architecture.

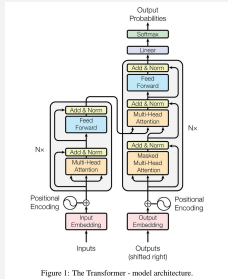
- Necesitamos una arquitectura de tipo transformer, de la cuál nos interesa el decoder.

¿Qué necesitamos para hacer un LLM?

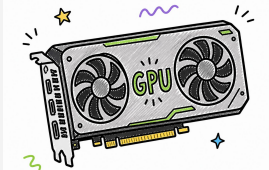
Muchos datos



Arquitectura Transformer



Pre-entrenamiento



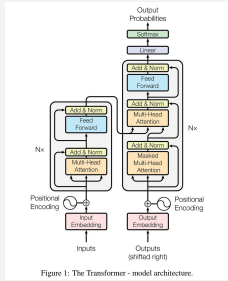
- Necesitamos hacer el pre-entrenamiento, que es: agarrar la arquitectura del modelo y entrenarla con los datos que recopilaste...
 - La entrenamos para poder predecir la siguiente palabra.
 - Le damos muchas oraciones diferentes y le pedimos que prediga la siguiente palabra y después verificamos los aciertos.
 - Con esto vamos ajustando el modelo ayudandolo a aprender cada vez más para mejorar en esa tarea.
 - Todo esto se hace con GPUs y necesitamos mucho poder de procesamiento para poder hacer estas tareas en paralelo y que sea más rápido.
 - Y necesitamos mucha plata para entrenar un modelo de este tipo... millones de dolares, además de qué lleva mucho tiempo, hasta meses de entrenamiento.

¿Qué necesitamos para hacer un LLM?

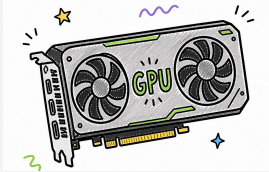
Muchos datos



Arquitectura Transformer



Pre-entrenamiento



reinforcement learning de humanos



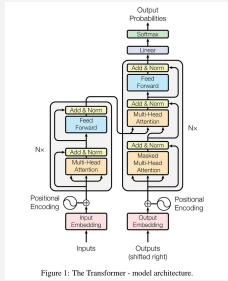
Una vez que tenemos el modelo pre-entrenado, podemos aplicar aprendizaje por refuerzo a partir de feedback humano.. básicamente tener humanos revisando las respuestas y generando ejemplos de cómo el modelo debería responder

¿Qué necesitamos para hacer un LLM?

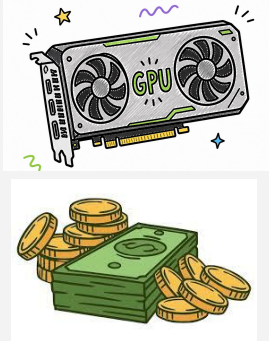
Muchos datos



Arquitectura Transformer



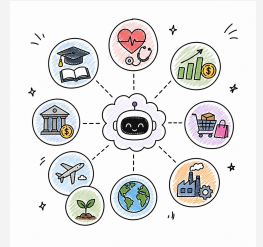
Pre-entrenamiento



reinforcement learning de humanos



fine-tuning (dominio específico)



Y por último, fine-tuning con datos específicos de un dominio según el caso de uso... Esto es lo que más les interesa a las empresas. Le damos una base de conocimiento interna propia y entonces los empleados y los clientes pueden tener mejores respuestas

Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?



Bien, entonces tenemos un LLM y le damos un *prompt*: “¿Cuál es la capital de Francia?”.

Todo ese texto entra como entrada al modelo. El modelo hace sus operaciones matemáticas y como salida produce **una sola palabra**, la siguiente más probable.

¿Cómo sabe cuál es la palabra más probable? Porque vio cantidades enormes de texto: internet, libros, Wikipedia, y mucho más. Entonces aprendió patrones sobre cómo las palabras suelen seguirse entre sí. Aunque no “recuerde” exactamente esa frase, puede predecir qué suele venir después basándose en todo lo que vio.

Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?

¿Cuál es la capital de Francia? La



En este caso, la palabra más probable podría ser “La”. Y esto es importante: **solo genera una palabra a la vez.**

Después, toma todo el input original más esa palabra generada (“La”) y lo vuelve a meter como entrada al modelo.

Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?

¿Cuál es la capital de Francia? La



En este caso, la palabra más probable podría ser “La”. Y esto es importante: **solo genera una palabra a la vez.**

Después, toma todo el input original más esa palabra generada (“La”) y lo vuelve a meter como entrada al modelo.

Cómo funciona un LLM: Predicción de la siguiente palabra

¿Cuál es la capital de Francia?
¿Cuál es la capital de Francia? La
¿Cuál es la capital de Francia? La **capital**



Otra vez hace los cálculos y genera la siguiente palabra más probable. Ahora podría ser “capital”. De nuevo, solo una palabra.

Luego repite el proceso: toma todo lo que tiene hasta ahora (“la capital”), lo usa como entrada y predice la siguiente palabra, por ejemplo “de”.

Cómo funciona un LLM: Predicción de la siguiente palabra

```
¿Cuál es la capital de Francia?  
¿Cuál es la capital de Francia? La  
¿Cuál es la capital de Francia? La capital  
¿Cuál es la capital de Francia? La capital de  
¿Cuál es la capital de Francia? La capital de Francia  
¿Cuál es la capital de Francia? La capital de Francia es  
¿Cuál es la capital de Francia? La capital de Francia es París.  
¿Cuál es la capital de Francia? La capital de Francia es París. <EOS>
```



Y así sigue, palabra por palabra, como si estuviera armando la frase paso a paso, hasta que en algún momento predice algo especial: el *token de fin de secuencia*.

Ese token indica que, según sus predicciones, la frase ya debería terminar. No lo vemos como usuarios, queda “detrás de escena”. Es simplemente una señal interna para que el modelo deje de generar texto.

En este ejemplo, el resultado final sería algo como: “La capital de Francia es París”.

La idea clave es esta: los modelos de lenguaje están diseñados para **predecir la siguiente palabra más probable**, una y otra vez, usando tanto el texto original como lo que ya generaron, en un proceso iterativo, hasta que deciden que la secuencia terminó.

POR QUÉ LOS LLMS ALUCINAN

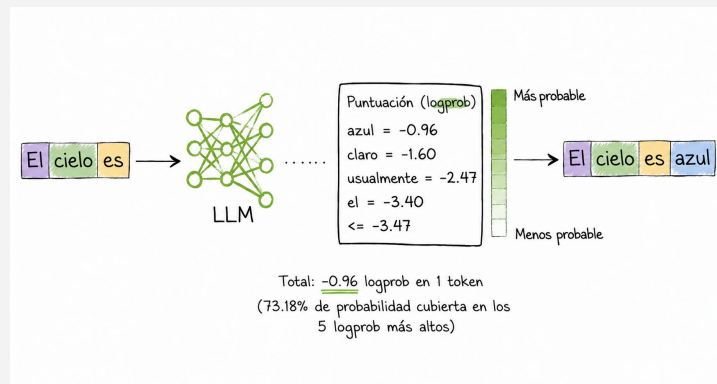
Y POR QUÉ ES INEVITABLE

140

por qué los LLMs alucinan y por qué es inevitable ?

¿Por qué los LLMs alucinan?

El modelo **no sabe si una afirmación es verdadera**
Sabe qué token es estadísticamente **más probable** según el contexto.



Los LLMs predicen el siguiente token más probable.

Va a tener una lista de todas las palabras del idioma en el que va a responder y cada palabra va a tener una probabilidad calculada. La que tenga la probabilidad más alta es el output.

El modelo no sabe si una afirmación es verdadera.

Sabe qué token es estadísticamente probable dado el contexto.

Una vez que entendés que el modelo predice tokens por probabilidad, entendemos que la alucinación es una consecuencia directa del mecanismo.

Si en su entrenamiento vio muchas veces “La Pampa no existe”, va a tender a generar esa afirmación con alta confianza, aunque no sea real. El modelo está completando un patrón, no verificando un hecho.

¿Por qué los LLMs alucinan?

El modelo no sabe si una afirmación es verdadera

Sabe qué token es estadísticamente más probable según el contexto.

cuantas letras r tiene strawberry?

Hay **dos** letras "r" en "strawberry".

La palabra se divide en "straw" y "berry":
una r en "straw" y una r en "berry".

También dijimos que los modelos funcionan con tokens... Porque un LLM no funciona exactamente como una persona leyendo texto carácter por carácter. Su "unidad mental" normalmente no son letras individuales, sino **tokens**.

Alucinaciones en el SOC

IOCs fabricados

El modelo puede generar hashes, IPs, dominios que *parecen* reales y son estadísticamente plausibles pero no corresponden a ningún artefacto real.

Puntos ciegos

El modelo no puede razonar sobre información posterior a su entrenamiento. Si le preguntás por contexto ocurrido después de su fecha de corte, va a inventar una respuesta.

Mucha confianza

Una respuesta que suena muy segura puede estar completamente equivocada.

La mitigación correcta no es "usar prompts más específicos": es nunca confiar en afirmaciones factuales del LLM sin verificación independiente contra fuentes reales (VirusTotal, tu SIEM, tu CMDB, inteligencia de amenazas verificada).

Entonces entendemos por qué los LLMs alucinan, pero hay entonces que entender que esto es importante sobretodo si vamos a resolver problemas en seguridad...

Hay algunos problemas que podemos encontrar... podemos tener:

- IOCs fabricados. El modelo puede generarte una IP, un hash, un dominio que parece completamente real, que tiene el formato correcto, que estadísticamente es plausible... pero que no existe en ningún reporte, en ninguna base de datos. Entonces no vamos a poder usarlos para reglas del SIEM por ejemplo.
- El segundo son los puntos ciegos. El modelo tiene una fecha de corte de entrenamiento. Si le preguntás por contexto de algo que ocurrió después de esa fecha, no te va a decir 'no sé' — te va a inventar una respuesta que suena convincente. Esto es especialmente crítico con amenazas nuevas.
- Y el tercero, que es quizás el más problemático: y es precisamente eso mismo... mucha confianza. El modelo no te dice 'estoy dudando'. Te responde con un tono seguro aunque esté equivocado. Una respuesta que suena como de un **senior** experto puede ser totalmente incorrecta.

La mitigación al problema de las alucinaciones no es escribir prompts más específicos... eso no resuelve el problema de fondo. La mitigación es nunca confiar

en afirmaciones del LLM sin verificarlas contra fuentes reales: tu SIEM, VirusTotal, tu CMDB etc

Qué no poner en un LLM externo

- Indicadores de compromiso activos de un incidente en curso (IPs, dominios, hashes de una brecha que todavía no fue divulgada)
- Logs con datos de usuarios internos (nombres, emails, actividad de empleados)
- Configuraciones de red, topología de infraestructura, rangos IP internos
- Código propietario de sistemas internos
- Datos de clientes o pacientes (PII, PHI)
- Cualquier información cubierta por NDA o regulaciones como GDPR o HIPAA

La alternativa correcta para entornos con datos sensibles es usar modelos que corren **localmente** (LLaMA, Mistral, Qwen en servidores propios) o proveedores con contratos enterprise que garanticen no usar tus datos para entrenamiento y que cumplan las regulaciones aplicables. El LLM corre en infraestructura propia y nunca sale un paquete a internet con contexto del incidente.

antes de ver qué no mandar a un LLM externo — quiero mencionar que hoy en día tenemos la opción de correr modelos localmente, en servidores propios. LLaMA, Mistral, Qwen son

modelos open source que pueden correr en su propia infraestructura. si bien no son modelos tan “inteligentes” cómo el último de openai o Anthropic.. el contexto del incidente nunca sale de su red y el tema de privacidad de datos desaparece.

Pero si están usando una API externa — OpenAI, Anthropic, Gemini — hay cosas que no deberían pasarle nunca...

- Primero, indicadores de compromiso activos de un incidente en curso. Si hay una brecha que todavía no fue divulgada públicamente y ustedes mandan esas IPs o esos hashes a una API externa, eso puede terminar en el dataset de entrenamiento del proveedor, o simplemente quedarse logueado en sus servidores.
- Segundo, logs con datos de usuarios internos: nombres, emails, actividad de empleados. Eso es información sensible que no debería salir del entorno.
- configuraciones de red e infraestructura: topología interna, rangos IP, porque al final esa es información que un atacante adoraría tener.
- También: código propietario de sistemas internos, datos de clientes o pacientes, y cualquier cosa cubierta por un NDA o regulaciones como GDPR

- o HIPAA.

Python para LLMs

```
import anthropic

client = anthropic.Anthropic(api_key="tu_api_key") # nunca hardcodeada - usá .env

response = client.messages.create(
    model="claude-3-opus-20240229",
    max_tokens=1024,
    temperature=0.2, # bajo para consistencia
    messages=[
        {"role": "user", "content": "Analiza esta alerta de seguridad: ..."}
    ]
)

print(response.content[0].text)
```

Cómo llaman a un LLM desde Python?

Este es el código mínimo para hacer una llamada a la API de Anthropic. Tienen el cliente, le pasan el modelo, el máximo de tokens, la temperatura — que la tenemos baja, en 0.2, para que las respuestas sean más consistentes y menos creativas — y el mensaje.

una cosa importante.... hay un comentario que dice 'nunca hardcodeada, usá .env' — la API key nunca va en el código.

Este es un ejemplo mínimo, pero porque sería importante que si queremos hacer llamadas a APIs reales, qué leamos la documentación de qué campos acepta, cómo es la respuesta qué devuelve y cómo parsearla..

Casos de uso de LLMs

Los LLMs son buenos en tareas donde el texto tiene patrones reconocibles y donde una respuesta aproximada es útil aunque no perfecta y siempre con verificación posterior.

Ejemplos:

- Triage de alertas
- Clasificación de tipo de eventos

```
prompt = f"""
You are a SOC analyst triaging security alerts.
Analyze this alert and provide:
1. Severity: CRITICAL/HIGH/MEDIUM/LOW
2. False positive likely: YES/NO/UNCERTAIN
3. Recommended action: INVESTIGATE_NOW/ESCALATE/MONITOR/CLOSE
4. Reasoning: explain your assessment in 2-3 sentences

Alert data:
{json.dumps(alerta_raw, indent=2)}
"""
```

```
prompt = f"""
Classify this security event into EXACTLY ONE category:
MALWARE_EXECUTION | DATA_EXFILTRATION | PRIVILEGE_ESCALATION |
LATERAL_MOVEMENT | CREDENTIAL_COMPROMISE | PHISHING |
POLICY_VIOLATION | SYSTEM_ERROR | LEGITIMATE_ACTIVITY

Event: {evento_texto}
"""
```

¿Para qué sirven los LLMs en seguridad concretamente?

La regla general es: son buenos en tareas donde el texto tiene patrones reconocibles y donde una respuesta aproximada es útil aunque no sea perfecta

Podemos usarlos para triage de alertas: pedirle que evalúe una alerta y devuelva la severidad, si el falso positivo es probable o no, acción recomendada, y el razonamiento. La respuesta de esto la podemos poner en un jira para que los analistas tengan la información más masticada...

también podemos usarlo para clasificación de tipo de evento. El modelo tiene que elegir exactamente una categoría de una lista fija: MALWARE_EXECUTION, DATA_EXFILTRATION, etcétera. Cuando le damos opciones cerradas, alucinamos mucho menos... pero aunque para clasificación de tipos convendría usar NLP clásico

importante: Esto no reemplaza al analista... el analista igual tiene que revisar. Pero si tienen 500 alertas y pueden pre-clasificarlas automáticamente y traer algo de info masticada ... mejor

Deobfuscación de código

Si le pasás al LLM un fragmento de código **ofuscado (intencionalmente difícil de leer)** y le pedís que lo analice, puede reconocer ciertos patrones y darte una idea general de qué está haciendo.

- sirve para acelerar la primera revisión
- luego decidir si hace falta investigarlo más a fondo en sandbox

Para malware muy sofisticado (exploits de zero-day, técnicas de stealth avanzadas), los LLMs pueden no ser confiables. El malware actual es frecuentemente más complejo que el texto de entrenamiento del LLM.

Un LLM no puede realmente ejecutar el malware para ver qué hace. Solo puede leer código estático. Malware puede tener comportamiento solo en ciertos contextos que el análisis estático no capture.

Los LLMs fueron entrenados hasta cierta fecha. Algo descubierto después de esa fecha, puede no ser reconocido.

un caso de uso interesante podría ser deobfuscación de código:

porque el malware viene ofuscado intencionalmente, código que hace lo mismo que código claro, pero que está escrito para que sea difícil de entender: variables sin nombre, strings codificadas en base64, funciones anidadas, etc.

Si le pasamos ese fragmento a un LLM y le pedimos que lo analice, puede reconocer ciertos patrones y darnos una idea general de qué está haciendo ese código. Sirve para acelerar la primera revisión y decidir si vale la pena mandarlo a un sandbox o descartarlo.

Pero hay limitaciones importantes. Para malware muy sofisticado — exploits de zero-day, técnicas de stealth avanzadas — el LLM puede quedarse corto o equivocarse, porque ese tipo de

malware es frecuentemente más complejo que cualquier cosa que haya visto en su entrenamiento.

También importante: el LLM no puede ejecutar el código. Solo lee texto.

El malware puede tener comportamientos que solo aparecen en ciertos contextos de ejecución, y eso el análisis del texto estático no lo captura.

nos sirve como primer filtro, no como veredicto final

Resumen de reportes

Cuando un reporte tiene 50 páginas, **un LLM puede resumirlo en una sola con lo esencial**: familia, vector de ataque, impacto, sistemas afectados, IOCs y acciones recomendadas.

- Se le da el reporte completo y se le pide una salida con estructura fija.

otro caso de uso es para resumen de reportes:

- Los reportes de threat intelligence pueden tener 50, 80, 100 páginas. Leerlos completos lleva horas. Un LLM puede resumirlo en una sola página con lo esencial: familia del malware, vector de ataque, impacto, sistemas afectados, IOCs, acciones recomendadas.

Resumen de reportes

Le pasás un reporte de malware, aunque venga en PDF o texto desordenado, y le pedís que extraiga y clasifique los IOCs.

- Puede identificar IPs, dominios, hashes, rutas de archivo, claves de registro y nombres de procesos, y devolver todo en JSON.
- Algo que antes hacía un analista a mano, ahora se resuelve en segundos y queda listo para usar en threat intel o reglas IDS.

Los LLMs pueden "inventar" IOCs que no existen realmente.

```
prompt = f"""
Extract all IOCs (Indicators of Compromise) from this malware report.

Categorize as:
- IP addresses (with country/ASN if available)
- Domain names
- File hashes (MD5, SHA1, SHA256)
- Registry keys modified
- Files created
- Network ports used
- Process names/behaviors
- URLs

Report:
{reporte_malware}

Format output as structured JSON:
{{
  "ips": [...],
  "dominios": [...],
  "hashes": [...],
  "ports": [...]
}}
```

En este prompt específico le estamos pidiendo que extraiga y clasifique los IOCs del reporte.

Le decimos exactamente qué categorías queremos — IPs con país y ASN si están disponibles, dominios, hashes por tipo, claves de registro modificadas, archivos creados, puertos, nombres de procesos, URLs. Y le pedimos la salida en JSON estructurado con un schema definido.

Cuando la salida es JSON con schema conocido, podemos parsearla programáticamente y usarla directamente — meterla en una lista de bloqueo, generar una regla, importarla en nuestro threat intel platform.

Algo que antes hacía un analista a mano en 2 horas, ahora se resuelve en segundos y queda listo para usar.

pero no nos olvidemos: los LLMs pueden inventar IOCs. El output siempre va a VirusTotal u otra fuente antes de usarse."

Generación de reportes de incidentes

- 1. EXECUTIVE SUMMARY (1 página)
 - Qué sucedió
 - Cuándo sucedió
 - Quién/Qué fue afectado
 - Impacto estimado
 - Recomendaciones
- 2. TIMELINE (1-2 páginas)
 - Eventos cronológicos desde primer indicador hasta contención
 - Acciones de respuesta tomadas
- 3. TECHNICAL ANALYSIS (2-5 páginas)
 - Detalles técnicos del incidente
 - IOCs encontrados
 - Métodos de ataque
 - Sistemas comprometidos
- 4. IMPACT ASSESSMENT (1-2 páginas)
 - Datos potencialmente expuestos
 - Sistemas afectados
 - Duración del acceso no autorizado
- 5. ROOT CAUSE (1 página)
 - Por qué sucedió el incidente
 - Vulnerabilidades explotadas
- 6. RECOMMENDATIONS (1-2 páginas)
 - Acciones para prevenir incidentes similares
- 7. APPENDICES
 - IOCs detallados
 - Logs de referencia
 - Screenshots

- Tenemos la estructura del reporte definida
- Pasarle toda la información que ya recolectó: el usuario afectado, los dispositivos, los IOCs que descubrió, la timeline de eventos, las acciones que tomó.

```
prompt = f"""
Generate a professional incident report based on the following case data.

Case Data:
{json.dumps(datos_caso, indent=2)}

Requirements:
- Write in formal, professional tone
- Include specific dates/times and technical details
- Organize chronologically
- Be comprehensive but concise
- Provide clear recommendations

Generate the full report with all sections:
[EXECUTIVE SUMMARY]
[TIMELINE]
[TECHNICAL ANALYSIS]
[IMPACT ASSESSMENT]
[ROOT CAUSE]
[RECOMMENDATIONS]
"""
```

otro caso de uso podría ser la generación de reportes de incidentes.

Cuando cierran un incidente, hay que escribir el reporte. Y los reportes de incidente tienen siempre la misma estructura: executive summary, timeline, análisis técnico, impact assessment, root cause, recomendaciones, apéndices con los IOCs. Siempre igual.

Si la estructura es siempre la misma, y nosotros ya recolectamos toda la información durante la investigación entonces el LLM puede redactar el reporte.

Lo que hubiera tomado 2-3 horas de redacción, ahora toma minutos.

Generación de reportes de incidentes

- Personalización por tipo de incidente

```
TEMPLATES = {
  "phishing": {
    "focus": ["email artifacts", "user actions", "email gateway logs"],
    "emphasis": "user education"
  },
  "malware": {
    "focus": ["process analysis", "network connections", "file hashes"],
    "emphasis": "endpoint isolation"
  },
  "data_exfiltration": {
    "focus": ["data accessed", "volume exfiltrated", "destinations"],
    "emphasis": "scope of exposure"
  },
  "insider_threat": {
    "focus": ["user actions", "data accessed", "timeline"],
    "emphasis": "behavioral patterns"
  }
}
```

```
tipo_incidente = "phishing"

prompt = f"""
Generate incident report for {tipo_incidente} incident.

Focus particularly on:
{' '.join(template['focus'])}

Emphasize in recommendations:
{template['emphasis']}

Case data:
{json.dumps(datos_caso)}
"""
```

Además, podemos generar prompts programáticamente... personalizarlo según el tipo de incidente.

| En este ejemplo tienen un diccionario de templates.

- Para phishing: el foco va en los artefactos de email y las acciones del usuario, y el énfasis en recomendaciones es educación del usuario.
- Para malware: foco en análisis de procesos, conexiones de red y hashes de archivos, énfasis en aislamiento del endpoint.
- Para exfiltración: foco en qué datos se accedieron, volumen y destinos, énfasis en scope del impacto.

El código selecciona el template según el tipo de incidente y lo inyecta en el prompt. Así el mismo sistema genera reportes que suenan distintos y relevantes para cada situación."

Generación de reportes de incidentes

Revisión humana y control de calidad del reporte: ¿Incluye todos los IOCs que encontraste? ¿La timeline está en orden? ¿Menciona al usuario afectado? ¿Tiene recomendaciones concretas?

Pero nosotros somos medio vagos: podemos hacer algunos chequeos automáticos!

```
def revisar_reporte(reporte_generado, datos_caso):  
    """Control de calidad del reporte generado"""  
  
    problemas = []  
  
    # Verificación 1: ¿Contiene datos específicos del caso?  
    if not datos_caso['usuario_afectado'] in reporte_generado:  
        problemas.append("Missing affected user name")  
  
    # Verificación 2: ¿Incluye IOCs?  
    for ioc in datos_caso['iocs']['urls']:  
        if ioc not in reporte_generado:  
            problemas.append(f"Missing IOC: {ioc}")  
  
    # Verificación 3: ¿Tiene recomendaciones?  
    if "recommend" not in reporte_generado.lower():  
        problemas.append("Missing recommendations section")  
  
    return {  
        "aprobado": len(problemas) == 0,  
        "problemas": problemas,  
        "requiere_revision": len(problemas) > 0  
    }
```

Y acá viene algo importante: revisión humana.

El reporte lo generó un LLM. Antes de mandarlo, alguien tiene que verificar: ¿incluye todos los IOCs que encontramos? ¿La timeline está en orden? ¿Menciona al usuario afectado? ¿Tiene recomendaciones concretas?

Pero — y acá viene lo bueno — como somos analistas y nuestro tiempo vale, podemos automatizar parte de esos chequeos. podemos hacer verificaciones automáticas:

ver si:

- el nombre del usuario afectado aparece en el reporte
- si Están todos los IOCs que teníamos
- si Hay una sección de recomendaciones



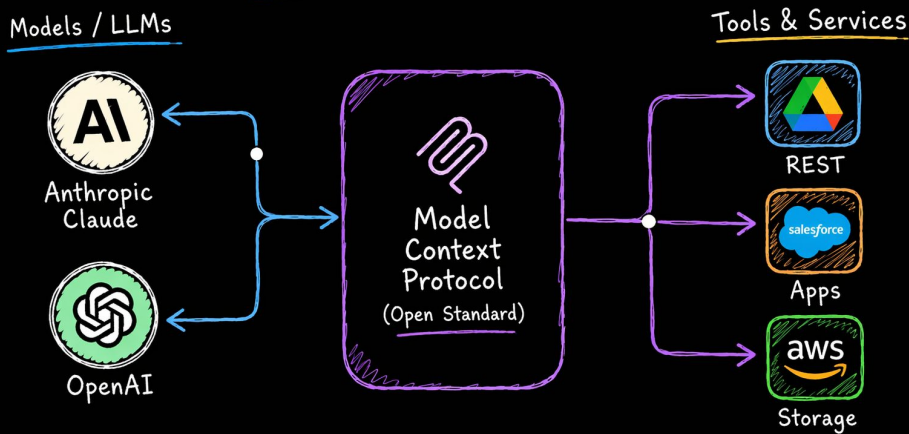
Los LLMs son una herramienta poderosa: aceleran el triage, resumen reportes, generan documentación. Pero el criterio, la verificación y la responsabilidad siguen siendo del analista.

Los LLMs son una herramienta poderosa para el SOC: aceleran el triage, resumen reportes, extraen IOCs y generan documentación. Pero son solo eso — una herramienta. El criterio, la verificación y la responsabilidad siguen siendo del analista.

MCP

Hola a todos! soy flor y bienvenidos de vuelta al curso de seguridad defensiva
hoy vamos a ver MCP en el SOC

Model Context Protocol



155

Durante años, los LLMs fueron herramientas de conversación: sabían mucho, pero no podían *hacer* nada. Podías preguntarle a Claude o ChatGPT qué hace un hash de malware, pero no podían consultarlo directamente en VirusTotal, ni buscar en tu SIEM, ni aislar un endpoint en tu EDR. El **Model Context Protocol** cambia eso.

MCP es un protocolo abierto que define cómo un LLM se conecta a herramientas, datos y sistemas externos de forma estandarizada. En términos prácticos: con MCP podés construir un asistente de seguridad que tiene acceso real a tu stack del SOC.

¿Qué problema resuelve MCP?

- Los LLMs solo pueden procesar texto
- Para actuar necesitan conectarse a herramientas externas
- Sin un estándar, cada integración es ad-hoc
- MCP es ese estándar

156

Un LLM por sí solo solo puede hacer una cosa: procesar texto y generar texto. No puede consultar tu SIEM, no puede buscar un hash en VirusTotal, no puede crear un ticket,

no puede aislar un host. Para hacer cualquiera de esas cosas necesita conectarse a herramientas externas.

Y acá aparece el mismo problema que vimos al principio con los conectores: si cada integración la hacemos a mano, de forma ad-hoc, terminamos con un LLM que habla con VirusTotal de una

manera, con el SIEM de otra, con Jira de otra. Cada conexión tiene su propia lógica, sus propios errores, su propio mantenimiento.

MCP — Model Context Protocol — es la solución a ese problema. Es un protocolo abierto, creado por Anthropic pero adoptado por la industria, que define exactamente cómo un LLM se comunica con herramientas externas. Un estándar único para todas las integraciones."

Arquitectura de MCP

- Tools: el LLM ejecuta acciones
- Resources: el LLM lee datos
- Prompts: plantillas reutilizables

157

┆ "MCP define tres primitivas. Tres tipos de cosas que un servidor MCP puede ofrecerle a un LLM.

┆ La primera son las tools — herramientas. Son acciones que el LLM puede ejecutar: consultar una API, escribir en una base de datos, mandar una alerta. Si los recursos son para leer, las tools son para hacer.

┆ La segunda son los resources — recursos. Son datos que el LLM puede leer: logs, tickets abiertos, configuraciones, el estado actual de un incidente. hay que pensarlo como un endpoint GET — solo lectura, solo contexto.

┆ La tercera son los prompts — plantillas. Son instrucciones reutilizables para tareas frecuentes. Por ejemplo: 'analizar esta alerta', 'generar reporte de incidente'. En lugar de reescribir el prompt

cada vez, lo definís una sola vez en el servidor y el agente lo reutiliza.

┆ Con estas tres primitivas podés construir integraciones muy complejas de forma ordenada."

MCP en seguridad: casos de uso

- Consultar reputación de un IOC en VirusTotal
- Buscar eventos relacionados en el SIEM
- Crear un ticket cuando se confirma un incidente
- Aislar un host en el EDR

┆ "¿Cómo se ve esto en un SOC concreto?"

┆ Imaginen un agente investigando una alerta. Encuentra una IP sospechosa — tiene una tool que consulta VirusTotal y trae el resultado directo al contexto. Encuentra eventos relacionados — tiene otra tool que busca en el SIEM sin que nadie tenga que abrir otra pestaña.

El analista confirma que es un incidente real — el agente tiene una tool que crea el ticket en Jira con toda la información ya cargada. Se decide aislar el host comprometido — el agente tiene una tool que ejecuta esa acción en el EDR.

┆ Todo dentro del mismo flujo. Sin cambiar de herramienta, sin copiar y pegar información entre sistemas. El agente orquesta todo y el analista toma las decisiones."



El MCP no le da inteligencia al LLM, le da manos.

159

Los LLMs ya son buenos razonando, analizando texto, identificando patrones. Lo que les faltaba era poder actuar sobre el mundo real. MCP es exactamente eso: la forma estándar de darle al LLM acceso a herramientas reales, con control humano donde importa.

Ahora vamos a ver esto en código.

Demo final: Pipeline SOC completo

`generate_data.py` → genera dataset en CSV / NDJSON / Parquet

`train_models.py` → entrena Isolation Forest + clasificador NLP

`demo.py` → corre el pipeline completo:

Paso 1: evento entrante (tráfico de red sospechoso)

Paso 2: Isolation Forest detecta anomalía

Paso 3: clasificador NLP identifica DATA_EXFILTRATION

Paso 4+5: LLM + MCP (tool-calling loop): enriquece evento, crea ticket

Paso 6: Human-in-the-Loop... analista aprueba o rechaza el aislamiento del host

Código en [github](#)

En esta clase vamos a dimensionar el problema del volumen de alertas en un SOC y por qué no escala operativamente.

Vamos a entender de dónde vienen muchos falsos positivos y cómo eso impacta en el equipo.

También vamos a poner en contexto el costo de no escalar bien los procesos de respuesta.

Y a partir de ahí, vamos a introducir criterios para decidir cuándo automatizar y cuándo no.

UN REPASO DE LO VISTO

Hola a todos! hoy vamos a ver IA en el SOC

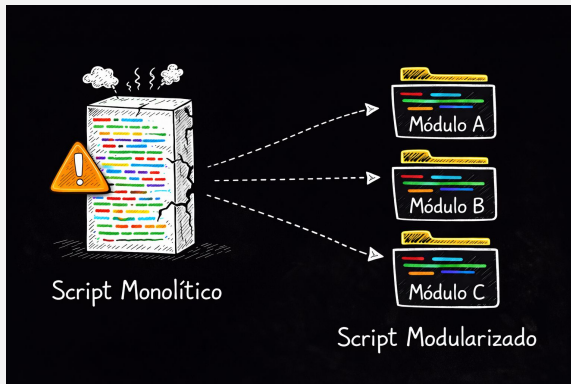
Automatización con Python

- Scripting en Python para tareas de seguridad
- Automatización de tareas repetitivas
- Buenas prácticas: modularidad, logging, testing básico



- Scripting en Python para tareas de seguridad
- Automatización de tareas repetitivas
- Buenas prácticas: modularidad, logging, testing básico

Integraciones y Conectores



- Desarrollo de conectores reutilizables
- Patrón ConectorBase
 - un fix beneficia a todos los conectores
- Patrón Registry
 - el código no sabe cómo se inicializó nada, solo pide lo que necesita
- Integración con APIs externas

- Desarrollo de conectores reutilizables
- Patrón ConectorBase
 - un fix beneficia a todos los conectores
- Patrón Registry
 - el código no sabe cómo se inicializó nada, solo pide lo que necesita
- Integración con APIs externas

SOAR y automatización de respuestas

- Runbooks y playbooks
- Orquestación de herramientas de seguridad
- Automatización de respuesta a incidentes

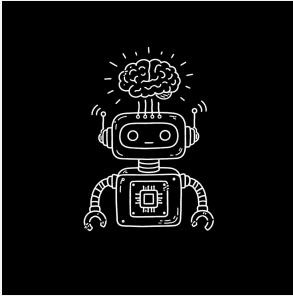
- Runbooks y playbooks
- Orquestación de herramientas de seguridad
- Automatización de respuesta a incidentes

ML en Seguridad y Detección de Anomalías

- Algoritmos comunes: árboles de decisión, Random Forest, Isolation Forest, SVM, redes neuronales, K-Means, DBSCAN, autoencoders
- Anomaly Detection: detectar lo que se desvía de lo normal
- UBA (User Behavior Analytics)
- UEBA: extiende UBA a entidades (servidores, routers, apps, IoT)
- UEBA + SIEM: reducir ruido, priorizar investigaciones

- Algoritmos comunes: árboles de decisión, Random Forest, Isolation Forest, SVM, redes neuronales, K-Means, DBSCAN, autoencoders
- Anomaly Detection: detectar lo que se desvía de lo normal
- UBA (User Behavior Analytics)
- UEBA: extiende UBA a entidades (servidores, routers, apps, IoT)
- UEBA + SIEM: reducir ruido, priorizar investigaciones

Introducción a IA y Machine Learning



- Qué es la IA y el ML; diferencia con reglas estáticas
- Aprendizaje supervisado vs. no supervisado
- Workflow: recolección → features → entrenamiento → evaluación → deploy
- Métricas: accuracy, precision, recall, F1
- Ajuste continuo del modelo (re-entrenamiento con feedback de analistas)
- Limitaciones del ML en seguridad: cold start, ataques adversariales, concept drift, garbage in/garbage out

- Qué es la IA y el ML; diferencia con reglas estáticas
- Aprendizaje supervisado vs. no supervisado
- Workflow: recolección → features → entrenamiento → evaluación → deploy
- Métricas: accuracy, precision, recall, F1
- Ajuste continuo del modelo (re-entrenamiento con feedback de analistas)
- Limitaciones del ML en seguridad: cold start, ataques adversariales, concept drift, garbage in/garbage out

NLP y LLMs en Seguridad

- NLP en seguridad: clasificar alertas, extraer IOCs, detectar patrones en texto
- Cómo funciona un LLM
- Alucinaciones: el modelo no valida hechos, predice tokens probables
- Riesgos: IOCs fabricados, puntos ciegos, exceso de confianza
- Qué no enviar a un LLM externo: IOCs activos, logs con PII, topología de red, código propietario
- Casos de uso: triage de alertas, clasificación de eventos, deobfuscación de código, resumen de reportes, generación de reportes de incidentes con estructura fija

- NLP en seguridad: clasificar alertas, extraer IOCs, detectar patrones en texto
- Cómo funciona un LLM
- Alucinaciones: el modelo no valida hechos, predice tokens probables
- Riesgos: IOCs fabricados, puntos ciegos, exceso de confianza
- Qué no enviar a un LLM externo: IOCs activos, logs con PII, topología de red, código propietario
- Casos de uso: triage de alertas, clasificación de eventos, deobfuscación de código, resumen de reportes, generación de reportes de incidentes con estructura fija

Model Context Protocol (MCP)

- Problema que resuelve: los LLMs solo procesan texto; MCP es el estándar para conectarlos a herramientas externas
- Arquitectura: Tools (el LLM ejecuta acciones), Resources (el LLM lee datos), Prompts (plantillas reutilizables)
- Casos de uso en seguridad: consultar reputación de IOCs en VirusTotal, buscar eventos en el SIEM, crear tickets de incidentes, aislar hosts en el EDR

- Problema que resuelve: los LLMs solo procesan texto; MCP es el estándar para conectarlos a herramientas externas
- Arquitectura: Tools (el LLM ejecuta acciones), Resources (el LLM lee datos), Prompts (plantillas reutilizables)
- Casos de uso en seguridad: consultar reputación de IOCs en VirusTotal, buscar eventos en el SIEM, crear tickets de incidentes, aislar hosts en el EDR

Muchas gracias!
PREGUNTAS?

Me pueden encontrar en

Mail: flor.p.vela@gmail.com

Github: [@florvela](#)



muchas gracias por haber llegado hasta aca, espero que hayan disfrutado de las clases y qué hayan aprendido algo nuevo y se lleven herramientas para jugar

Aca les dejo de nuevo mi contacto por si quedaron dudas o si no fui clara en algo
espero leer sus mensajes